

LLM Inference & Optimization Interview Q&A Cheatsheet

Real Measured Results from This Topic's Own Executed Notebooks,
Grounded in Every Module's Hand Calculations

Target Persona: Senior AI Engineer / Applied AI Engineer (~3 YOE)

Focus Areas: Prefill/decode fundamentals, KV cache & GQA/MQA, PagedAttention, FlashAttention, inference quantization, batching strategies, speculative decoding, serving architecture, production monitoring & cost modeling

Includes: Conversational + Deep-Dive Answers, Key Formulas, Production Trade-offs, Common Mistakes, Follow-Up Q&As, Final Revision Sheet

LLM Inference & Optimization – Top 54 Interview Questions & Answers

1. Inference Fundamentals & the Autoregressive Decoding Loop (Q1–Q6)

Question 1: Why is autoregressive decoding inherently sequential, and how does that sequential dependency shape inference's batching structure and prefill/decode behavior differently from training?

[ESSENTIAL]

Conversational Answer

"Each new token's computation depends on every token generated before it — the model has to know what it just produced before it can decide what comes next. That's the real, structural reason decoding can't be parallelized across the sequence the way training can: at training time you already have the full target sequence, so every position's loss can be computed in one parallel pass. At inference time, position $t+1$ genuinely doesn't exist yet until position t has been sampled. That single fact ripples through everything about how inference systems are built differently from training systems: training optimizes for throughput over huge, fixed batches processed over hours; inference has to serve many real, independent, sequentially-generating requests at once, each one only able to advance one token at a time, and batching has to happen *across* requests rather than within a single request's own generation."

Intuitive Example

- Writing a sentence one word at a time, where each new word has to make sense given every word you've already committed to, versus grading a hundred already-finished essays in parallel — you can grade all hundred at once, but you can't write word 8 of one sentence before word 7 exists.

Key Interview Points

- **Real dependency:** token $t+1$'s computation genuinely requires token t to already exist.
- **Training vs. inference:** training has the full target sequence upfront (parallelizable across positions); inference does not.

- **Batching axis shifts:** inference batches *across* independent requests, not within one request's own sequential generation.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a structural/architectural distinction, not a quantitative one.

Production Perspective & Trade-offs

This sequential dependency is the root cause behind nearly every technique this topic covers: KV caching exists to avoid re-deriving already-computed positions, batching strategies exist to keep the GPU busy across many sequential streams at once, and speculative decoding exists to get more real useful work out of each expensive verification pass.

Common Mistakes

1. Assuming a "smarter batching algorithm" alone can make a single sequence's own decoding parallel — the sequential dependency is architectural, not a scheduling inefficiency.
2. Conflating training's parallelism (across sequence positions, with the full target known) with inference's parallelism (across independent requests only).

COMMON FOLLOW-UP QUESTIONS

Q: Does this sequential constraint apply to prefill too?

No — prefill processes all real prompt tokens in one parallel pass, since they're all already known upfront; the sequential constraint applies specifically to decode, where each new token is genuinely unknown until sampled.

Q: Could a model architecture avoid this constraint entirely?

Non-autoregressive generation architectures exist and sidestep it, but at a real, current cost to output quality/coherence for open-ended generation — autoregressive decoding remains dominant precisely because the sequential dependency is what lets each token condition on everything genuinely already committed.

One-Line Takeaway

Takeaway: Decode is sequential because each token's computation genuinely requires the previous token to exist — inference has to batch across requests, not within one.

Question 2: Walk through the roofline model — what does arithmetic intensity measure, and what does the ridge point represent?

[ESSENTIAL]

Conversational Answer

"Arithmetic intensity is the ratio of real compute (FLOPs) to real data movement (bytes) a given operation requires — how much math you get to do per byte you had to fetch. The ridge point is a property of the specific hardware: it's the arithmetic intensity at which a GPU's peak compute throughput and peak memory bandwidth are equally the bottleneck. Below that ridge point, an operation can't keep the compute units fed fast enough — it's memory-bandwidth-bound, and real performance is capped by how fast bytes arrive, not by how fast the math could run. Above the ridge point, there's more compute work per byte than the memory system needs to sustain, so real performance is capped by the compute units instead. The whole point of the roofline model is that it tells you *which* lever is worth pulling — optimizing compute for a memory-bound operation, or vice versa, wastes real engineering effort."

Intuitive Example

- A kitchen where one chef can chop ingredients faster than the delivery truck can bring them, versus a kitchen with a mountain of ingredients and only one slow chef — in the first case, faster delivery helps; in the second, hiring a second chef does.

Key Interview Points

- **Arithmetic intensity:** FLOPs / bytes moved — how much compute per byte of real data transferred.
 - **Ridge point:** the hardware's peak-FLOPs/peak-bandwidth ratio — the intensity where compute and bandwidth are equally limiting.
 - **Diagnostic purpose:** tells you whether an operation is memory-bandwidth-bound or compute-bound, and therefore which optimization lever actually helps.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$I = \frac{\text{FLOPs}}{\text{Bytes moved}} \quad I_{\text{ridge}} = \frac{\text{Peak FLOPs/s}}{\text{Peak Bandwidth}}$$

An operation with $I < I_{\text{ridge}}$ is memory-bandwidth-bound; $I > I_{\text{ridge}}$ is compute-bound.

Production Perspective & Trade-offs

Every later optimization in this topic targets one side of this: FlashAttention and quantization reduce real memory traffic (helping memory-bound operations); batching increases real arithmetic intensity by amortizing weight reads across more concurrent work.

Common Mistakes

1. Optimizing FLOPs (e.g., a "more efficient" algorithm with fewer operations) for a workload that's actually memory-bandwidth-bound, where the real bottleneck was never compute in the first place.
2. Treating the ridge point as a fixed, universal number rather than a real, hardware-specific value that changes across GPU generations and models.

COMMON FOLLOW-UP QUESTIONS

Q: Does a higher-FLOPs GPU always mean faster inference?

Only for compute-bound operations — for a real memory-bandwidth-bound workload (a common decode-phase pattern), a GPU with more peak FLOPs but the same real memory bandwidth won't help, since bandwidth was already the binding constraint.

Q: How would you determine a specific GPU's real ridge point?

Divide its real, published peak FLOPs/s by its real peak memory bandwidth — both are standard, publicly documented hardware specs for a given GPU model.

One-Line Takeaway

Takeaway: Arithmetic intensity is FLOPs per byte moved; the ridge point is the hardware-specific threshold separating memory-bandwidth-bound from compute-bound — and it tells you which lever is worth pulling.

Question 3: Given peak FLOPs/s and peak memory bandwidth, compute a GPU's ridge point, then compute and classify decode's and prefill's arithmetic intensity at a given model size and prompt length.

[ESSENTIAL]

Conversational Answer

"Walking through Module 01's own real, verified numbers: for an illustrative GPU with 312 TFLOPs/s peak compute and 2039 GB/s peak bandwidth, the ridge point is $312e12 / 2039e9 \approx 153$ FLOPs/byte. For a 7-billion-parameter model at FP16, one decode step reads roughly all 7B parameters (14GB of

weight traffic) to produce just one new token — that gives an arithmetic intensity of exactly 1.0 FLOPs/byte, far below the 153 ridge point, so decode lands squarely in the memory-bandwidth-bound region. Prefill processing 512 real prompt tokens in one pass amortizes that same weight-read cost across 512 tokens instead of 1, pushing arithmetic intensity up to roughly 512 FLOPs/byte — above the ridge point, landing prefill in the compute-bound region. Same model, same hardware, genuinely different regimes, purely because of how many tokens are processed per weight read."

Intuitive Example

- Reading an entire cookbook (the weights) to cook one dish (decode) is wildly inefficient per-dish; reading the same cookbook once to cook 512 dishes in a row (prefill-style batching) amortizes that fixed reading cost far better.

Key Interview Points

- **Ridge point:** $312\text{e}12/2039\text{e}9 \approx 153$ FLOPs/byte, from real illustrative hardware specs.
 - **Decode:** $I \approx 1.0$ FLOPs/byte — memory-bandwidth-bound, well below the ridge point.
 - **Prefill (512 tokens):** $I \approx 512$ FLOPs/byte — compute-bound, above the ridge point.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$I_{\text{decode}} = \frac{2N_{\text{params}}}{N_{\text{params}} \times \text{bytes}_{\text{dtype}}} = \frac{2}{\text{bytes}_{\text{dtype}}} \quad I_{\text{prefill}} = \frac{2N_{\text{params}} \times n_{\text{tokens}}}{N_{\text{params}} \times \text{bytes}_{\text{dtype}}} = \frac{2 n_{\text{tokens}}}{\text{bytes}_{\text{dtype}}}$$

Production Perspective & Trade-offs

This is the real, quantitative basis for the entire prefill/decode-differentiated-optimization strategy this topic builds on — the two phases genuinely warrant different techniques because they sit in different roofline regions at typical batch sizes.

Common Mistakes

1. Forgetting that decode's arithmetic intensity is independent of model size — it's always $2/\text{bytes}_{\text{dtype}}$ per sequence, since both FLOPs and bytes scale identically with parameter count.
2. Assuming this specific 153/1.0/512 result generalizes to every model size and hardware combination without recomputing for the actual real configuration in question.

COMMON FOLLOW-UP QUESTIONS

Q: Why doesn't decode's arithmetic intensity depend on how many parameters the model has?

Because both the numerator (FLOPs, $2N_{\text{params}}$) and denominator (bytes, $N_{\text{params}} \times \text{bytes}_{\text{dtype}}$) scale linearly with N_{params} , so it cancels out — decode's real intensity is fixed by precision alone (2 FLOPs/byte at FP16), independent of model size.

Q: What happens to decode's real arithmetic intensity under batching?

A real batch of B concurrent decode steps amortizes the same weight read across B sequences at once, pushing real arithmetic intensity toward $I \approx 2B/\text{bytes}_{\text{dtype}}$ — this is exactly the mechanism explored in Question 4 below.

One-Line Takeaway

Takeaway: At FP16, decode sits at $I \approx 1.0$ (memory-bandwidth-bound) and 512-token prefill sits at $I \approx 512$ (compute-bound) against a real ridge point of ≈ 153 — the same model, genuinely different regimes.

Question 4: Why is "decode is memory-bandwidth-bound, prefill is compute-bound" described as a typical pattern rather than a universal rule? What real workload change can shift decode's regime?

[ESSENTIAL]

Conversational Answer

"Because the arithmetic intensity numbers above assumed a batch size of one decode step at a time — real production serving almost never runs that way. If you batch many real concurrent decode steps together, each one still needing the same weights read, that weight read gets amortized across the whole batch, exactly like prefill amortizes it across prompt tokens. Module 01's own reference code verified this directly: batching 256 concurrent decode steps together pushed the real arithmetic intensity up to 256 FLOPs/byte — above the same 153 ridge point that a single decode step (at $I=1.0$) sat far below. So 'decode is memory-bound' is genuinely the typical pattern at low concurrency, but it's a workload-dependent claim, not a law of nature — enough real concurrent decode requests can shift the *same* per-token computation into the compute-bound region."

Intuitive Example

- Reading a recipe once to cook one dish is wasteful; reading the same recipe once to cook 256 identical dishes simultaneously amortizes that fixed reading cost dramatically — the "recipe-reading-bound" framing stops applying once you're cooking at scale.

Key Interview Points

- **Typical, not universal:** the memory-bound decode claim holds at low real batch sizes/concurrency, not unconditionally.
 - **Real verified shift:** 256 concurrent decode steps pushed arithmetic intensity to $I=256$, above the ridge point.
 - **Same mechanism as batching (Module 06):** amortizing a fixed weight-read cost across more concurrent work.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$I_{\text{batched_decode}} \approx \frac{2B}{\text{bytes}_{\text{dtype}}}$$

where B is the number of concurrent decode steps sharing one weight read.

Production Perspective & Trade-offs

This is a real, direct link between Module 01's roofline framing and Module 06's batching strategies: high-concurrency serving genuinely changes which bottleneck dominates, which is part of why continuous batching's throughput gains are real and substantial, not just a scheduling nicety.

Common Mistakes

1. Reciting "decode is always memory-bound" as an absolute fact in an interview without the batch-size qualifier — a real, common overclaim this topic explicitly corrects.
2. Assuming a shift into the compute-bound region under batching means decode's real latency-per-token stops mattering — it still does; the *bottleneck type* shifted, not the importance of decode-phase optimization.

COMMON FOLLOW-UP QUESTIONS

Q: At what real batch size does decode's arithmetic intensity cross the ridge point?

Solving $2B/\text{bytes}_{\text{dtype}} > I_{\text{ridge}}$ for B at FP16 (bytes= 2) and $I_{\text{ridge}} \approx 153$ gives $B > 153$ — consistent with the real 256-batch verification exceeding it.

Q: Does this mean batching always eliminates the memory-bandwidth bottleneck?

It shifts the **arithmetic-intensity classification**, but real KV-cache memory traffic still grows with batch size too (Module 02) — a large batch can still hit a real memory-**capacity** wall even after crossing the compute/bandwidth ridge point.

One-Line Takeaway

Takeaway: "Decode is memory-bound" is the typical low-concurrency pattern, not a universal rule — real batching (256 concurrent steps, verified) can push the same computation past the ridge point into compute-bound territory.

Question 5: Precisely distinguish TTFT and TPOT — what does each one's cost actually scale with?

[ESSENTIAL]

Conversational Answer

"TTFT — time to first token — is dominated by the real prefill pass: processing the entire prompt once before the first output token can be produced. Its real cost scales with prompt length, since prefill's FLOPs grow with the number of prompt tokens processed. TPOT — time per output token — is the real, per-step decode cost: how long each subsequent token takes once generation is underway. Its real cost is driven by the fixed cost of reading model weights (and the KV cache) at every single step, largely independent of how many tokens have already been generated, at least until the KV cache itself grows large enough to matter. These are genuinely different real bottlenecks with genuinely different real scaling behavior, which is exactly why they get tracked as two separate metrics rather than folded into one 'latency' number."

Intuitive Example

- TTFT is like the setup time before a factory line starts running — it scales with how much has to be set up (the prompt); TPOT is the steady per-unit time once the line is running, largely constant regardless of how many units have already come off it.

Key Interview Points

- **TTFT:** dominated by prefill, real cost scales with prompt length.
- **TPOT:** dominated by per-step decode, real cost is largely fixed per step (weight + KV cache read).
- **Different bottlenecks:** prefill is typically compute-bound, decode is typically memory-bandwidth-bound (Question 4's caveat still applies).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula beyond Question 3's arithmetic-intensity expressions — TTFT and TPOT are the real, observable latency consequences of prefill's and decode's respective roofline positions.

Production Perspective & Trade-offs

Real production SLOs typically track TTFT and TPOT separately (Module 09) precisely because they respond to different real levers — reducing prompt length or improving compute throughput helps TTFT; reducing memory traffic per step (quantization, GQA/MQA) or amortizing weight reads (batching) helps TPOT.

Common Mistakes

1. Reporting a single blended "average latency" number that obscures whether a real regression came from TTFT (prefill-side) or TPOT (decode-side) — the fix for each is different.
2. Assuming TPOT is perfectly constant across a whole generation — it stays *roughly* flat (Notebook 01's real finding: 7.8% spread), not exactly, as the real KV cache does grow with sequence length.

COMMON FOLLOW-UP QUESTIONS

Q: Which metric matters more for a user-facing chat interface?

Real user-perceived responsiveness depends on both — TTFT determines how long a user waits before anything appears, while TPOT determines how quickly text streams in afterward; a real product often has separate SLOs for each rather than one combined target.

Q: Does reducing prompt length always reduce TTFT proportionally?

Only while prefill stays in whichever real roofline regime it started in — since prefill is typically compute-bound, real TTFT scales close to proportionally with prompt length in that regime, but this is again a typical pattern, not a guarantee independent of the real workload.

One-Line Takeaway

Takeaway: TTFT scales with real prompt length (prefill-dominated); TPOT is a largely fixed per-step cost (decode-dominated) — genuinely different bottlenecks tracked as separate metrics for a reason.

Question 6: A real notebook measured TTFT staying nearly flat across a 32x range of prompt lengths, while TPOT stayed flat across generation length — yet converting TTFT to a per-prompt-token cost revealed a real $\approx 897x$ gap

versus TPOT. Why does the per-token conversion matter for interpreting this result correctly?

[ESSENTIAL]

Conversational Answer

"Raw TTFT values alone were genuinely misleading on their own: real measured TTFT only moved from 143.64ms at 32 prompt tokens to 157.61ms at 1024 tokens — a 32x range in prompt length producing barely any change in raw TTFT. Taken at face value, that could look like prefill cost 'doesn't scale with length,' which isn't the real story. Dividing TTFT by prompt length gives the real *per-token* prefill cost, and that number dropped sharply as prompt length grew — down to about 0.15ms per prompt token at 1024 tokens. Compared against the real measured TPOT floor of about 138ms per decode token, that's a real, measured $\approx 897\times$ gap. The raw TTFT curve looked flat because this specific small model, on this specific GPU, wasn't yet compute-saturated at these prompt lengths — the per-token view is what actually isolates and reveals prefill's real efficiency advantage, exactly the qualitative pattern Module 01's roofline framing predicts."

Intuitive Example

- A delivery truck that takes roughly the same total time whether it's carrying 1 package or 30 looks "flat" by total-trip-time — but the real per-package efficiency tells a completely different, much more informative story.

Key Interview Points

- **Raw TTFT looked flat:** 143.64ms $\rightarrow$ 157.61ms across a real 32x prompt-length range.
 - **Per-token conversion reveals the real story:** cost-per-prompt-token dropped to $\approx 0.15\text{ms/token}$ at 1024 tokens.
 - **Real measured gap:** $\approx 897\times$ cheaper per-token for prefill than the real measured TPOT floor ($\approx 138\text{ms/token}$).
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Cost per prompt token} = \frac{\text{TTFT}}{N_{\text{prompt_tokens}}}$$

This is the real, direct metric that isolates prefill's per-token efficiency from its raw, less-informative total latency.

Production Perspective & Trade-offs

This is a genuine, real caution about metric selection in production monitoring too — a raw latency number can look stable while masking a real, meaningful trend that only appears once normalized per unit of real work (tokens, requests, etc.).

Common Mistakes

1. Reading a flat raw-latency curve as "no real scaling relationship exists" without checking whether normalizing by workload size reveals a different, real story.
2. Generalizing this specific $\approx 897\times$ figure to other models/hardware without re-measuring — it's a real result for this specific small model and consumer GPU, not a universal constant.

COMMON FOLLOW-UP QUESTIONS

Q: Why might raw TTFT stay nearly flat even though real per-token cost is dropping sharply?

At this small model size and these prompt lengths, real fixed per-call overhead (kernel launch, Python dispatch) is plausibly a non-trivial fraction of the measured milliseconds, and prefill isn't yet saturating this GPU's real available compute — both real, honest factors the notebook names rather than hides.

Q: Would this per-token gap look different on a much larger model?

Plausibly smaller in relative terms if fixed per-call overhead becomes negligible next to a much larger model's genuine per-token compute cost — this specific ratio is a real, hardware/model-specific measurement, not a claimed universal figure.

One-Line Takeaway

Takeaway: A flat raw TTFT curve can hide a real, large per-token efficiency story — normalizing by prompt length revealed a genuine $\approx 897\times$ prefill-vs-decode per-token cost gap that raw TTFT alone did not.

2. KV Cache Mechanics & Memory Management (Q7–Q12)

Question 7: What does the KV cache store, and why does caching it mean only the new token's key/value needs to be computed at each step, while attention still reads the full cached prefix?

[ESSENTIAL]

Conversational Answer

"At every transformer layer, attention needs a key vector and a value vector for every token in the sequence so far. Those key/value vectors for already-generated tokens never change once computed — token 5's key and value don't depend on what token 12 turns out to be. The KV cache stores those vectors, per layer, per token, exactly once. That means at each new decode step, the model only has to compute the *new* token's own key and value — it doesn't need to recompute anything for tokens already in the cache. Attention itself still has to read the *entire* cached prefix to compute the new token's attention scores against every prior position — caching doesn't shrink how much gets read for attention, it eliminates redundantly recomputing the keys/values themselves at every step."

Intuitive Example

- Keeping page-by-page notes on a book you're summarizing means writing today's page's notes is fast — you don't re-read and re-summarize every prior page — but you still flip back through all your prior notes each time to keep the summary consistent.

Key Interview Points

- **What's cached:** per-layer key/value vectors for every already-generated token — fixed once computed.
 - **What caching saves:** recomputing prior tokens' keys/values — only the new token's K/V get computed each step.
 - **What caching does not shrink:** attention still reads the full cached prefix to compute the new token's scores.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula in this question specifically — the memory-footprint formula for the cache itself is Question 8/9's focus.

Production Perspective & Trade-offs

This distinction matters precisely because it explains *why* KV cache memory (not recomputation FLOPs) becomes the real production bottleneck at scale — the compute savings are real and significant, but they shift the real constraint onto memory capacity and bandwidth instead (Questions 8-11).

Common Mistakes

1. Describing the KV cache as reducing the *amount of data attention reads* — it reduces *recomputation*, not the real per-step attention read volume.
2. Assuming KV cache eliminates decode's memory-bandwidth cost entirely — it eliminates redundant recomputation FLOPs, but real memory traffic (reading weights and the growing cache) remains the dominant per-step cost.

COMMON FOLLOW-UP QUESTIONS

Q: Does the KV cache change what attention computes, mathematically?

No — the real attention output is mathematically identical with or without caching; caching is a real, pure computational-efficiency optimization, not a change to the underlying attention mechanism itself.

Q: What happens to the KV cache when a sequence finishes?

Its real memory should be freed/reclaimed so it can be reused for other requests — how efficiently and quickly that reclamation happens is exactly what Module 03's PagedAttention addresses.

One-Line Takeaway

Takeaway: The KV cache eliminates redundant recomputation of prior tokens' keys/values — it doesn't shrink how much attention reads, which is exactly why cache *memory* becomes the real bottleneck instead.

Question 8: Walk through the KV-cache memory footprint formula — why does it depend on N_{KV_heads} specifically, not the total query-head count?

[ESSENTIAL]

Conversational Answer

"In standard multi-head attention, every query head has its own dedicated key/value head, so the KV-head count and query-head count are the same number and the distinction doesn't matter. But Grouped-Query Attention and Multi-Query Attention deliberately break that equality — multiple query heads share a smaller number of real key/value heads. The KV cache only ever stores keys and values, never queries, so its real memory cost depends specifically on how many *distinct KV heads* exist, not on

how many query heads attend to them. Using the total query-head count in the formula would overstate real KV-cache memory for any GQA/MQA model — which, as it turns out, is most modern production models, including the real model this topic's own notebooks used."

Intuitive Example

- A library with 14 reading desks (query heads) that all share access to only 2 actual bookshelves (KV heads) — the shelving space needed depends on the 2 shelves, not the 14 desks reading from them.

Key Interview Points

- **Only K/V get cached:** queries are never stored, so cache size depends on KV-head count alone.
 - **GQA/MQA break the query=KV-head equality:** multiple query heads share fewer real KV heads.
 - **Using query-head count overstates real memory** for any GQA/MQA architecture.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Mem}_{\text{KV}} = 2 \times B \times L \times N_{\text{layers}} \times N_{\text{KV_heads}} \times d_{\text{head}} \times \text{bytes}_{\text{dtype}}$$

Production Perspective & Trade-offs

This is exactly why GQA/MQA have become standard in modern production models — the real memory savings from fewer KV heads compound directly with both batch size and sequence length, making long-context/high-concurrency serving genuinely tractable.

Common Mistakes

1. Using a model's total attention-head count (from a generic architecture diagram) instead of checking its real, specific `num_key_value_heads` config — a real, common source of memory-estimation error for any GQA/MQA model.
2. Assuming GQA/MQA are training-time-only choices — they directly, multiplicatively affect real inference-time memory, which is exactly this question's point.

COMMON FOLLOW-UP QUESTIONS

Q: How would you find a real deployed model's actual N_{KV_heads} ?

Check its real, published config (e.g., `num\_key\_value\_heads` in a Hugging Face `config.json`) rather than assuming it equals the query-head count — Notebook 02 did exactly this and found a real, genuine GQA config (2 KV heads vs. 14 query heads) on the model it used.

Q: Does MQA ($N_{KV_heads} = 1$) always beat GQA on real memory savings?

Yes, MQA gives the real maximum possible KV-head reduction for a given query-head count, but at a real, separate cost to model quality that GQA's intermediate KV-head count is specifically designed to balance against.

One-Line Takeaway

Takeaway: KV-cache memory depends on the number of real key/value heads, not query heads — GQA/MQA exploit that distinction to cut memory directly and multiplicatively.

Question 9: Given a model's real layer count, KV-head count, and head dimension, compute its KV-cache memory footprint at a given batch size and sequence length, once under MHA and once under GQA.

[ESSENTIAL]

Conversational Answer

"Walking through Module 02's own real, verified hand calc: for an illustrative 32-layer model with $d_{head}=128$ at FP16, standard MHA with 32 query heads (so $N_{KV_heads}=32$ too) gives a per-token, per-batch-item cost of $2 \times 32 \times 32 \times 128 \times 2 = 524,288$ bytes — 512KB per token. At a real high-concurrency workload, batch size 32 and sequence length 4096, that's $512KB \times 4096 \times 32 = 64.0$ GB of real KV-cache memory under MHA. Switching to a real GQA configuration with only 8 KV heads — a 4x reduction from 32 — cuts that to exactly 16.0 GB at the identical workload, a real, direct 4x memory reduction matching the 4x KV-head reduction exactly. Both numbers were verified against a real model's own executed config in Notebook 02, not just hand-computed."

Intuitive Example

- Same warehouse, same amount of inventory arriving — but organizing it across 8 loading docks instead of 32 uses proportionally less total dock space per unit of goods, exactly tracking the dock-count reduction.

Key Interview Points

- **MHA, 32 KV heads, B=32, L=4096:** 64.0 GB real computed KV-cache memory.
 - **GQA, 8 KV heads, same workload:** 16.0 GB — exactly a 4x reduction, matching the KV-head ratio.
 - **Real, verified:** Notebook 02 confirmed an exact byte-for-byte match against this same formula using a real model's real config.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same formula as Question 8, evaluated at two different N_{KV_heads} values holding everything else fixed — the reduction factor between the two results always equals the ratio of N_{KV_heads} values exactly, since every other formula term is linear and unchanged.

Production Perspective & Trade-offs

This exact-ratio property is a real, useful mental shortcut in production capacity planning — the KV-cache memory savings from switching MHA to GQA at a fixed KV-head reduction factor are fully predictable without re-deriving the whole formula each time.

Common Mistakes

1. Forgetting to hold batch size and sequence length fixed when comparing MHA vs. GQA memory — the comparison is only meaningful at matched workload conditions.
2. Assuming binary-GB (GiB, $\div 1024^3$) and decimal-GB ($\div 10^9$) give the same number — Module 02's own hand calc caught and fixed exactly this real unit-mismatch bug during development.

COMMON FOLLOW-UP QUESTIONS

Q: Why did Module 02's development process specifically flag a units bug here?

An initial hand calc mixed decimal-GB and binary-GB units, producing a wrong intermediate figure (68.7GB instead of the correct 64.0GB) — caught only because the reference code's own assertion against a real computed value failed, a genuine example of verification catching a real, easy-to-make error.

Q: Would this same 4x reduction hold at a different batch size or sequence length?

Yes — the reduction factor is determined purely by the N_{KV_heads} ratio (here, $32/8 = 4$), independent of batch size or sequence length, since both cancel identically in the MHA-vs-GQA ratio.

One-Line Takeaway

Takeaway: At a real, verified high-concurrency workload, GQA's 4x-fewer KV heads gave exactly a real 4x KV-cache memory reduction — 64.0 GB (MHA) down to 16.0 GB (GQA), matched byte-for-byte in Notebook 02.

Question 10: Why is "KV cache dominates memory usage" workload-dependent rather than universally true — under what conditions does KV cache overtake model weights?

[ESSENTIAL]

Conversational Answer

"Whether KV cache or model weights dominate real GPU memory is genuinely a function of the real workload, not a fixed property of the model. Model weights are a real, fixed cost paid once, regardless of traffic. KV cache scales with *both* batch size and sequence length simultaneously — so at low concurrency and short context, that fixed weight cost dominates; at high concurrency or long context, KV cache's cost grows to meet and exceed it. Module 02's own hand calc showed this concretely: at low concurrency (batch 1, 512 tokens), KV cache was a real 0.25GB against 13.04GB of real weight memory — weights clearly dominant. At high concurrency (batch 32, 4096 tokens) under MHA, KV cache reached 64.0GB — nearly 5x the same 13.04GB of weights. Same model, same weights, genuinely different real answer depending on the workload."

Intuitive Example

- A restaurant's fixed kitchen equipment cost is the dominant expense on a slow night with few tables, but on a packed night the real, per-table ingredient cost can add up to dwarf that fixed equipment cost — same kitchen, different dominant cost depending on how busy it is.

Key Interview Points

- **Weights:** a real, fixed memory cost, independent of traffic.
- **KV cache:** scales with batch size *and* sequence length together — grows with real concurrency/context.
- **Real crossover:** low concurrency/short context → weights dominate; high concurrency/long context → KV cache dominates (Module 02's own verified numbers).

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Compare Question 8's KV-cache formula's output directly against $N_{\text{params}} \times \text{bytes}_{\text{dtype}}$ (real weight memory) at the actual real workload in question — there's no single universal answer without plugging in real numbers.

Production Perspective & Trade-offs

This workload-dependence is exactly why real capacity planning has to be done against the *actual expected* concurrency and context-length distribution, not just against model size — a system sized only for weights can hit a genuine, unexpected memory wall the moment real traffic or context length grows.

Common Mistakes

1. Sizing a production deployment's GPU memory budget purely off model weight size, ignoring real KV-cache growth under the actual expected traffic pattern.
2. Treating the specific crossover point from one real hand calc (a particular model size, batch size, sequence length) as a universal threshold rather than something that has to be recomputed for the actual real deployment in question.

COMMON FOLLOW-UP QUESTIONS

Q: What two real levers most directly determine where the crossover happens?

Real concurrency (batch size) and real context length (sequence length) — both multiply directly into the KV-cache formula, while weight memory stays constant regardless of either.

Q: Does GQA/MQA change where the crossover happens?

Yes, directly — Question 9's real 4x KV-cache reduction under GQA pushes the crossover point to a real, higher concurrency/context-length threshold than the same workload would hit under standard MHA.

One-Line Takeaway

Takeaway: Whether weights or KV cache dominate memory is workload-dependent — Module 02's own real hand calc showed weights dominant at low concurrency/short context and KV cache dominant (nearly 5x weights) at high concurrency/long context, same model either way.

Question 11: A real notebook measured a model's exact KV-cache tensor size (matching the formula byte-for-byte) but found `memory_allocated()`'s delta

was $\approx 26\times$ larger. What real, distinct source accounted for nearly all of that gap, and why does this matter for capacity planning?

[ESSENTIAL]

Conversational Answer

"Notebook 02 measured two genuinely different things and found they diverged sharply. First, it extracted the real, exact KV-cache tensor sizes directly from the model's own `past_key_values` object — that matched Module 02's formula exactly, byte-for-byte, at every sequence length tested. Second, it measured the real *delta* in `torch.cuda.memory_allocated()` across the same forward passes — and that delta came out roughly $26\times$ larger than the real KV-cache size. Root-causing it precisely: the model's real logits tensor — shape (batch, sequence length, vocabulary size), and this model's vocabulary is 151,936 tokens — accounted for 93.9% to 96.1% of that gap across the tested sequence lengths. `memory_allocated()`'s delta wasn't measuring KV cache at all, mostly — it was measuring the real logits activation tensor, an entirely separate real memory consumer with nothing to do with the cache."

Intuitive Example

- Trying to measure how much water one specific faucet added to a bathtub by watching the total tub water level rise — but a second, much bigger faucet was running into the same tub the whole time, so the total rise mostly reflects the second faucet, not the one you meant to measure.

Key Interview Points

- **Exact KV-cache tensor size:** matched Module 02's formula byte-for-byte at every real tested sequence length.
 - `memory_allocated()` **delta:** $\approx 25.7\times$ - $26.3\times$ larger than the real KV-cache size.
 - **Root cause, precisely identified:** the real logits tensor (batch $\times$ seq_len $\times$ 151,936 vocab) accounted for 93.9%-96.1% of that gap.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Logits tensor size $\approx B \times L \times V \times \text{bytes}_{\text{dtype}}$ — this scales with vocabulary size V , a real, separate quantity from any of Question 8's KV-cache formula terms.

Production Perspective & Trade-offs

This is a real, concrete, quantitatively-verified caution against using a single blunt memory-allocator reading as a proxy for any one specific real memory consumer — production memory monitoring

needs to isolate the actual component of interest (KV cache specifically), not a conflated allocator-level delta.

Common Mistakes

1. Using `memory_allocated()`'s raw delta across a forward pass as "the KV-cache memory cost" — a real, demonstrated $\approx 26\times$ overstatement in this exact case.
2. Assuming this specific $\approx 26\times$ figure generalizes to every model — it's driven by this specific real model's large vocabulary size relative to its small hidden dimension, and would differ for a model with a different vocab-to-hidden-size ratio.

COMMON FOLLOW-UP QUESTIONS

Q: How did the notebook get a real, trustworthy KV-cache-only measurement instead?

By extracting the real key/value tensors directly from the model's own ``past_key_values`` object and summing their exact real byte sizes (`numel() × element_size()``) — a ground-truth measurement independent of whatever else the CUDA allocator happened to also be tracking.

Q: Would this gap be smaller for a model with a much smaller vocabulary?

Plausibly — the logits tensor's real size scales directly with vocabulary size, so a smaller-vocabulary model would contribute proportionally less real activation memory relative to the same KV-cache size, narrowing the real gap.

One-Line Takeaway

Takeaway: A real $\approx 26\times$ gap between measured KV-cache tensor size and `memory_allocated()`'s delta was traced precisely to the logits tensor, not the cache — a concrete, verified reason not to use a blunt allocator reading as a component-specific proxy.

Question 12: How do GQA and MQA reduce KV-cache memory without proportionally reducing model quality?

[ESSENTIAL]

Conversational Answer

"GQA and MQA only reduce the number of distinct key/value *projections* — the query heads, and the model's real representational capacity for computing attention *scores* against those keys, are completely unaffected. Empirically, having a group of query heads share a smaller number of real KV heads costs surprisingly little real model quality while directly, multiplicatively cutting the KV cache's memory footprint. The intuition is that the queries still get to specialize independently — they're what

determines *what* each head attends to — while sharing the underlying keys/values mostly affects the *representation* being attended to, which turns out to tolerate real sharing well in practice."

Intuitive Example

- Several analysts (query heads) each independently deciding what to look for, but pulling from a smaller shared set of source documents (KV heads) instead of each having their own private copy — the analysis quality holds up because each analyst's own judgment (query) is untouched, only the shared source material changed.

Key Interview Points

- **What's shared:** only key/value projections — queries remain fully independent per head.
 - **Why quality holds up:** real attention *scoring* flexibility (queries) is preserved; only the shared representation (KV) is reduced.
 - **Real trade-off spectrum:** GQA (moderate sharing) to MQA (maximal sharing, $N_{KV_heads}=1$) trade progressively more real memory savings against progressively more real quality risk.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this is the qualitative mechanism behind Question 8/9's quantitative memory-reduction results.

Production Perspective & Trade-offs

This is exactly why GQA specifically (rather than full MQA) has become the real, common production default — it captures most of the real memory-savings benefit while keeping the quality risk more moderate than MQA's maximal sharing.

Common Mistakes

1. Assuming any KV-head reduction is quality-neutral by default — it's an empirical trade-off that real model architects tune and validate, not a free lunch guaranteed by the mechanism alone.
2. Confusing GQA/MQA's real KV-head sharing with reducing the number of query heads — query-head count and real attention-head diversity are unaffected by this technique.

COMMON FOLLOW-UP QUESTIONS

Q: Is GQA/MQA a choice made at training time or can it be applied to an already-trained model?

It's fundamentally a real architectural choice baked in at training time — retrofitting it onto an already-trained standard MHA model would require real, additional fine-tuning to adapt the model to the new, reduced KV-head structure.

Q: Does this topic's own real notebook model use GQA or MHA?

A real, genuine GQA config — Notebook 02 confirmed the model's real, extracted config has 14 query heads sharing only 2 real KV heads, a 7x sharing ratio, directly grounding Module 02's formula in a real, currently-deployed architecture choice rather than only an illustrative example.

One-Line Takeaway

Takeaway: GQA/MQA only reduce shared key/value projections, leaving query-head diversity untouched — which is why they cut real KV-cache memory multiplicatively while empirically costing little real model quality.

3. PagedAttention & Memory-Efficient Serving (Q13–Q18)

Question 13: Precisely state what PagedAttention does and does not do — why is "it compresses the KV cache" a common, incorrect claim?

[ESSENTIAL]

Conversational Answer

"PagedAttention improves how KV-cache memory is *allocated, utilized, and shared* across a serving system's requests. It does not shrink the real per-token KV-cache cost — that's Module 02's territory (GQA/MQA) and Module 05's territory (quantization). What PagedAttention fixes is a completely different real problem: naive contiguous allocation reserves a worst-case-length block of memory for every sequence up front, and most of that reservation typically sits real, allocated, and unused for the sequence's whole lifetime. PagedAttention allocates in small, fixed-size blocks on demand instead — the exact same real amount of KV data ends up stored either way, but far less memory sits reserved-and-wasted alongside it."

Intuitive Example

- Renting a parking garage floor per car "just in case" it needs to park a truck someday, versus assigning parking spots as vehicles actually arrive — the same number of real vehicles get parked either way, but the second approach wastes far less real garage space.

Key Interview Points

- **What it does:** improves real allocation, utilization, and sharing of existing KV-cache memory.
 - **What it does not do:** shrink the real per-token KV-cache size — the underlying data volume is unchanged.
 - **Common misconception:** calling it "compression" — it's an allocation-efficiency technique, not a size-reduction technique.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this question is architectural/conceptual, matching Module 03's own treatment.

Production Perspective & Trade-offs

The real payoff is indirect but substantial: better utilization means more real concurrent sequences fit in the same fixed GPU memory budget, which directly raises real effective batch size and therefore real throughput — without touching the per-token cost Module 02's formula governs.

Common Mistakes

1. Claiming PagedAttention reduces the real total memory a model needs to serve requests — it reduces *wasted* over-reservation, not the real underlying KV data volume.
2. Conflating PagedAttention with quantization or GQA/MQA as competing memory-reduction techniques — they operate on genuinely different real levers and compose together rather than substituting for each other.

COMMON FOLLOW-UP QUESTIONS

Q: If PagedAttention doesn't reduce KV-cache size, what's its real, measurable benefit?

A real, direct reduction in **wasted** reserved-but-unused memory — Module 03's own worked example showed this concretely (Question 15).

Q: Does PagedAttention require any model-architecture changes?

No — it's purely a serving-system memory-management technique, orthogonal to the model's own architecture (unlike GQA/MQA, which is a real training-time architectural choice).

One-Line Takeaway

Takeaway: PagedAttention improves KV-cache allocation and utilization — it does not compress or shrink the real per-token KV-cache cost, a common, worth-correcting misconception.

Question 14: Walk through the contiguous-allocation waste problem — why does reserving for the worst case waste real memory even when most real sequences finish early?

[ESSENTIAL]

Conversational Answer

"Without a paging scheme, a serving system that wants to guarantee it never runs out of room for a growing sequence has to reserve a contiguous block sized for the maximum sequence length the system supports — for *every* sequence, since it can't know in advance how long that specific sequence will actually run. Most real sequences finish well short of that maximum. The gap between what's reserved and what's actually used sits there, real and allocated, for the sequence's entire lifetime, doing nothing useful — and directly limiting how many other real concurrent sequences can fit in the same fixed memory budget."

Intuitive Example

- Booking an entire week-long hotel stay "just in case" for every guest, even though most guests only stay one or two nights — the reserved-but-empty room-nights add up fast across many guests.

Key Interview Points

- **Why worst-case reservation happens:** no way to know a sequence's real final length in advance without a paging mechanism.
 - **Real waste source:** the gap between reserved maximum and actual real usage, sitting idle for the sequence's whole lifetime.
 - **Compounding effect:** this waste multiplies across every concurrent sequence, directly capping real effective batch size.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — the quantitative version is Question 15's worked example.

Production Perspective & Trade-offs

This is the real, direct trade-off contiguous allocation forces: over-provision (real, measurable waste) or under-provision (real risk of running out of room mid-generation and having to reject or evict an in-flight sequence) — paging removes that trade-off entirely by allocating on demand.

Common Mistakes

1. Assuming a "smarter" contiguous-allocation heuristic (e.g., predicting sequence length in advance) fully solves this — real prediction is imperfect, and paging sidesteps the need for prediction altogether.
2. Treating this as a memory-*capacity* problem alone rather than also an allocation-*policy* problem — the real underlying data isn't the issue, the reservation strategy is.

COMMON FOLLOW-UP QUESTIONS

Q: What's the real alternative to over-provisioning that doesn't require prediction?

Block-based (paged) allocation — reserve memory incrementally, in small fixed units, only as a sequence actually grows, per Question 13.

Q: Does this waste problem apply equally to prefill and decode?

It applies to the real KV cache regardless of phase — both prefill's initial cache-build and decode's ongoing cache-growth need real memory reserved for the sequence's eventual (unknown in advance) full length under a contiguous scheme.

One-Line Takeaway

Takeaway: Contiguous allocation reserves worst-case-length memory per sequence because it can't predict real final length in advance — and that reserved-but-unused gap is real, substantial waste that directly caps concurrent capacity.

Question 15: Given a set of real variable-length sequences, a maximum supported length, and a block size, compute the real wasted-memory percentage under contiguous vs. paged allocation.

[ESSENTIAL]

Conversational Answer

"Module 03's own real, verified worked example: five sequences with real lengths $\{120, 340, 890, 45, 600\}$ — actually used tokens totaling 1,995 — against a maximum supported length of 2,048. Under contiguous allocation, every sequence reserves the full 2,048 tokens regardless of its real length: $2,048 \times 5 = 10,240$ total reserved slots, against 1,995 real used, giving 8,245 wasted slots — about 80.5% waste. Under paged allocation with a 16-token block size, each sequence only reserves $\lceil L/16 \rceil$ blocks — the total reserved comes to 2,032 slots, giving only 37 wasted slots, about 1.8% waste. That's a real, computed $\approx 44\times$ reduction in wasted memory, verified by direct code execution, not just estimated."

Intuitive Example

- Booking exactly the number of nights each guest actually needs (rounded up to the nearest half-week block) instead of a full month for every guest — the wasted room-nights shrink dramatically.

Key Interview Points

- **Contiguous:** 10,240 reserved, 1,995 used → 8,245 wasted (≈80.5%).
 - **Paged (block=16):** 2,032 reserved, 1,995 used → 37 wasted (≈1.8%).
 - **Real reduction factor:** ≈44x less wasted memory, verified via direct code execution.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Contiguous waste} = (\text{max_len} \times n_{\text{sequences}}) - \sum L_i \quad \text{Paged waste} = \sum (\lceil L_i / \text{block_size} \rceil - 1) \times \text{block_size}$$

Production Perspective & Trade-offs

This ≈44x reduction is workload-specific to this exact example's real length distribution and block size — the real, general point is that paged waste is *bounded per-sequence* (Question 16), while contiguous waste grows with how far below the maximum a real sequence's length falls.

Common Mistakes

1. Treating the specific ≈44x figure as a universal constant rather than a result of this exact worked example's real length distribution, maximum length, and block size.
2. Forgetting that paged allocation's real waste is never exactly zero — the last, partially-filled block of each sequence still carries some real internal fragmentation (Question 16).

COMMON FOLLOW-UP QUESTIONS

Q: How was this real 44x figure verified, not just computed by hand?

Module 03's reference code directly executed both allocation formulas against the same real length list and asserted the computed waste percentages and reduction factor matched the hand calc exactly — a real, passing verification, not just arithmetic on paper.

Q: What would happen to paged waste with a much larger block size (e.g., 128 instead of 16)?

Real per-sequence waste would grow (bounded by block_size – 1 per sequence, per Question 16) — a real, direct trade-off between fewer allocation-bookkeeping operations (larger blocks) and more internal fragmentation (also larger blocks).

One-Line Takeaway

Takeaway: On this real worked example, contiguous allocation wasted $\approx 80.5\%$ of reserved memory versus paged allocation's $\approx 1.8\%$ — a real, code-verified $\approx 44\times$ reduction in wasted memory.

Question 16: Under a simple block-based allocation model, why is paged allocation's per-sequence internal-fragmentation waste bounded by the block size, while contiguous allocation's waste is not similarly bounded regardless of the maximum supported sequence length?

[ESSENTIAL]

Conversational Answer

"Under this simplified block-based model, each sequence only ever wastes space in its *last*, partially-filled block — at most $(\text{block_size} - 1)$ tokens' worth of real internal fragmentation, no matter how long the sequence's full maximum supported length is. Contiguous allocation has no equivalent bound: its real waste for a given sequence is $(\text{maximum supported length} - \text{actual real length})$, which can be arbitrarily large as the maximum supported length grows, completely independent of block size since contiguous allocation has no blocks at all. That's the real structural reason paging's waste stays small and predictable while contiguous allocation's waste scales with how generous the system's real worst-case length ceiling is."

Intuitive Example

- Filling water into fixed-size cups (blocks) always wastes at most one partially-filled cup's worth per pour, regardless of how big the overall container the water might theoretically need to fill someday is — versus reserving one giant tank sized for the theoretical maximum every single time.

Key Interview Points

- **Paged model's real bound:** at most $(\text{block_size} - 1)$ wasted tokens per sequence, from its one partially-filled final block.
 - **Contiguous model's lack of bound:** $\text{waste} = (\text{max_len} - \text{actual_len})$, scaling with how large max_len is set.
 - **Explicit assumption:** this bound holds under the simplified block-allocation model used here — real production allocators may add their own bookkeeping overhead on top.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Per-sequence paged waste} = \lceil L/\text{block_size} \rceil \times \text{block_size} - L < \text{block_size}$$

Production Perspective & Trade-offs

This bound is exactly why real production block sizes are tuned empirically — smaller blocks tighten the real per-sequence waste bound further but add more real per-block bookkeeping overhead, a genuine trade-off Module 03 names explicitly.

Common Mistakes

1. Stating this bound as a universal, tool-independent guarantee rather than a property of this specific, simplified block-allocation model — a real production allocator's actual overhead can differ.
2. Assuming a smaller block size is strictly better without weighing the real bookkeeping-overhead cost that comes with more, smaller blocks.

COMMON FOLLOW-UP QUESTIONS

Q: What's the real trade-off in picking an extremely small block size?

Tighter real per-sequence waste bound, but more real per-block management overhead (more blocks to track per sequence) — production engines tune this empirically rather than minimizing block size purely theoretically.

Q: Does this bound change if a sequence's real final length is much longer than expected?

No — the bound stays at $(\text{block_size} - 1)$ regardless of how long the sequence actually runs, since waste only ever accumulates in the one currently-partially-filled final block.

One-Line Takeaway

Takeaway: Under a simple block-allocation model, paged waste per sequence is bounded by $(\text{block_size} - 1)$ regardless of maximum supported length — contiguous allocation has no such bound, since its waste scales directly with how generous that maximum is set.

Question 17: How does block-based allocation directly enable KV-cache sharing (e.g., for beam search or shared prefixes) in a way contiguous allocation cannot?

[ESSENTIAL]

Conversational Answer

"Because each block is an independently addressable unit drawn from a shared pool, multiple sequences that happen to share an identical prefix — beam search candidates branching from the same partial sequence, or several requests sharing a common system prompt — can have their own sequence-specific block tables simply *point at* the same underlying real blocks, instead of each sequence duplicating that data in its own private memory region. Contiguous allocation has no equivalent: each sequence's memory is one single private reserved region, with no structural way for two sequences to reference the same underlying bytes."

Intuitive Example

- Multiple readers checking out the same library book by reference (a shared block) instead of each reader needing their own personal photocopy of the entire book (private contiguous memory).

Key Interview Points

- **Block addressability:** each block in the shared pool can be referenced by multiple sequences' own block tables.
 - **Real use cases:** beam search (shared prefix across candidates), shared system prompts across multiple real requests.
 - **Contiguous allocation's structural limitation:** one private region per sequence, with no mechanism to reference shared underlying data.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is an architectural capability, not a quantitative claim.

Production Perspective & Trade-offs

Real prefix sharing directly compounds with the allocation-efficiency benefit from Question 15 — a shared system prompt across many concurrent real requests doesn't just avoid duplication once, it avoids it *per request*, a real, substantial multiplier at production scale.

Common Mistakes

1. Assuming sharing works automatically for any two sequences with similar (but not identical) content — real block-level sharing requires an actual identical prefix at the token level, not merely similar text.
2. Overlooking that shared blocks need real, careful reference-counting/copy-on-write handling once sequences diverge — a real implementation detail production serving engines have to get right.

COMMON FOLLOW-UP QUESTIONS

Q: What happens to a shared block once two sequences that referenced it start to diverge?

The real serving engine needs a copy-on-write-style mechanism — the shared block stays shared for the identical prefix, and each sequence gets its own real, private block(s) once its content actually diverges from the others.

Q: Does this sharing capability reduce real per-token KV-cache cost the way GQA/MQA does?

No — it avoids real *duplication* across sequences that happen to share content, a genuinely different real lever from GQA/MQA's per-token cost reduction (Question 8), and the two compose together rather than substituting for each other.

One-Line Takeaway

Takeaway: Block-based allocation lets multiple sequences reference the same underlying real memory for an identical shared prefix — a real sharing capability contiguous allocation's one-private-region-per-sequence design structurally cannot offer.

Question 18: A real notebook fed its own measured, straggler-containing generation lengths (not synthetic numbers) into a paged-vs-contiguous simulation and found a $\approx 6.4x$ waste reduction. Why is it important that this stayed labeled a simulation even though its inputs were genuinely real?

[ESSENTIAL]

Conversational Answer

"Notebook 05 took its own real, measured generation lengths from an actual batch of real model generations — including two genuine straggler sequences that never hit a stop token — and fed those exact real numbers into Module 03's contiguous-vs-paged waste formula. That produced a real-data-grounded result: 68.75% contiguous waste down to 10.71% paged waste, a $\approx 6.4x$ reduction. But the allocation logic itself was still a plain Python accounting exercise — no actual PagedAttention memory manager, no actual multi-request GPU serving system ran. Using real inputs makes it a *simulation*

grounded in real data, not an actual implementation or a real A/B benchmark of PagedAttention-vs-contiguous serving — and that distinction matters because conflating the two would overstate what was actually demonstrated."

Intuitive Example

- Running real, measured traffic-count data through a paper traffic-flow model to estimate congestion reduction from a new road design — genuinely useful and grounded in real numbers, but still not the same claim as having actually built and measured the new road.

Key Interview Points

- **Real inputs:** genuine measured generation lengths from an actual batch run, including two real stragglers.
 - **Simulated mechanism:** the allocation waste computation itself was a Python accounting exercise, not a real memory manager.
 - **Why the distinction matters:** real-data-grounded $\neq$ a real implementation or a real A/B serving benchmark — overstating this would misrepresent what was actually shown.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same formulas as Question 15, evaluated on real (not synthetic) measured length inputs — the formula and its interpretation are unchanged; only the provenance of the input data differs.

Production Perspective & Trade-offs

This distinction is a real, general discipline worth carrying into any production analysis: grounding a model or simulation in genuinely real measured inputs makes its output more credible than a purely synthetic example, but it still isn't the same evidentiary weight as an actual deployed A/B test.

Common Mistakes

1. Describing this result as "we benchmarked PagedAttention" — no real PagedAttention implementation was involved; only its allocation *logic* was simulated.
2. Dismissing the result as "just synthetic" because the mechanism was simulated — the inputs were genuinely real, which is a meaningfully stronger grounding than a fully synthetic example, even though the mechanism itself wasn't.

COMMON FOLLOW-UP QUESTIONS

Q: Where did these real straggler lengths originally come from?

The same notebook's own real batched-generation experiment (Question 36's subject) — two of eight real sequences ran the full token budget without emitting a stop token, directly carrying that real finding into this simulation's inputs.

Q: What would it take to turn this into a real A/B benchmark instead of a simulation?

Actually deploying and measuring a real PagedAttention-based serving system (e.g., vLLM) against a real contiguous-allocation baseline under the same real workload — infeasible in this environment per the topic's own stated hardware constraints, which is exactly why the simulation route was chosen and labeled honestly instead.

One-Line Takeaway

Takeaway: Real measured generation lengths grounded this simulation's inputs, but the allocation mechanism itself remained simulated — a real, worth-preserving distinction from an actual PagedAttention implementation or A/B benchmark.

4. FlashAttention & IO-Aware Attention Computation (Q19–Q24)

Question 19: Why is FlashAttention described as an IO-aware optimization rather than a FLOP-reduction technique?

[ESSENTIAL]

Conversational Answer

"Naive attention computes the exact same real mathematical result as FlashAttention — the FLOP count for exact attention is essentially unchanged between the two. What FlashAttention actually targets is how much data moves between the GPU's slow, off-chip HBM and its fast, on-chip SRAM during that computation. Naive attention materializes the full attention score matrix and writes/reads it to/from HBM multiple times during softmax normalization and the final weighted sum; FlashAttention processes attention in small tiles that stay in fast on-chip memory, accumulating the result incrementally, so that large intermediate matrix never gets written to slow HBM at all. Same math, same answer, dramatically less real memory traffic — that's precisely what 'IO-aware' means here."

Intuitive Example

- Doing a multi-step calculation entirely on a small whiteboard next to you versus writing every intermediate result to a filing cabinet down the hall and walking back and forth to fetch it each

time — the final answer is identical, but the real time spent moving between locations is not.

Key Interview Points

- **Same FLOPs:** exact attention's real compute requirement is essentially unchanged.
 - **Different IO:** FlashAttention avoids materializing the full score matrix in slow HBM.
 - **"IO-aware" precisely means:** optimizing real memory traffic, not the underlying mathematical operation count.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula here — the simplified HBM-access-count comparison is Question 21's focus.

Production Perspective & Trade-offs

This distinction directly explains why FlashAttention's real benefit shows up specifically in wall-clock latency and peak memory, not in a reduced FLOP count reported by a profiler — a real, common point of confusion this question exists to correct.

Common Mistakes

1. Describing FlashAttention as making attention "mathematically more efficient" in a FLOP-count sense — the real efficiency gain is entirely on the memory-traffic side.
2. Conflating FlashAttention's IO optimization with an algorithmic approximation of attention — it computes the exact same real result, not an approximation.

COMMON FOLLOW-UP QUESTIONS

Q: Would FlashAttention help on a hypothetical GPU with infinite memory bandwidth?

Real benefit would shrink toward zero — its entire value proposition is reducing real memory-bandwidth-bound traffic; a hardware constraint that no longer exists removes the problem it solves.

Q: Does FlashAttention change attention's real output values at all?

No — it's a real, exact reformulation of the same computation (using online softmax accumulation), not an approximation; its output should match naive attention's output up to real, ordinary floating-point precision differences.

One-Line Takeaway

Takeaway: FlashAttention targets real HBM traffic, not FLOPs — same math, same answer, dramatically less real data movement between slow and fast memory.

Question 20: Walk through why tiled attention avoids materializing the full attention score matrix in HBM, and how that reduces real HBM↔SRAM traffic compared to naive attention — without relying on a single universal asymptotic formula to make the point.

[ESSENTIAL]

Conversational Answer

"Naive attention computes the full $N \times N$ score matrix — every query against every key — and has to write that whole matrix out to slow HBM, then read it back in for softmax normalization, then read it again to multiply against V . Tiled attention instead processes small blocks of Q , K , and V at a time, entirely within fast on-chip SRAM: it computes each block's partial contribution and updates a running, online softmax accumulator incrementally, so the full score matrix is never assembled anywhere in slow memory — only small tiles ever exist at once. The real, direct consequence is that HBM traffic scales with how much of Q , K , V , and the output actually needs to move in and out — not with the size of an intermediate matrix that tiling avoids ever fully materializing."

Intuitive Example

- Adding up a huge column of numbers by keeping a running total in your head as you go, rather than writing every single partial sum down on paper and re-reading the whole page each time you add the next number.

Key Interview Points

- **Naive attention's real IO cost:** writing and re-reading the full materialized score matrix multiple times.
 - **Tiled attention's mechanism:** small blocks processed entirely in fast SRAM, with an online (running) softmax accumulator.
 - **Real consequence:** the large intermediate matrix never exists in slow HBM at all — traffic scales with $Q/K/V$ /output movement, not with a materialized intermediate.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No universal asymptotic formula asserted here — Module 04's own simplified access-count comparison (Question 21) is explicitly labeled an intuition-level illustration, not the paper's exact IO-complexity proof.

Production Perspective & Trade-offs

Real production kernels (FlashAttention-2/3) tune tile sizes to the specific target GPU's actual SRAM capacity — the qualitative avoiding-materialization mechanism holds generally, but the exact real traffic count depends on tuned, hardware-specific implementation details beyond this question's scope.

Common Mistakes

1. Asserting a specific asymptotic complexity (e.g., " $O(N^2)$ " vs. " $O(N \cdot d)$ ") as an exact, universal law rather than a simplified, intuition-building comparison — real production kernel traffic also depends on tile size relative to SRAM capacity.
2. Describing "online softmax" as an approximation — it's a real, exact mathematical reformulation that produces identical results to standard softmax, just computed incrementally.

COMMON FOLLOW-UP QUESTIONS

Q: What real GPU resource makes tiling possible in the first place?

Fast, on-chip SRAM — small in capacity but far faster than HBM; tiling is specifically sized to fit within it, exploiting the real speed differential between the two memory tiers.

Q: Is the exact real IO-complexity proof something a candidate should be able to derive in an interview?

Not typically — the qualitative mechanism (avoiding full-matrix materialization) is the real, expected interview-level understanding; the paper's exact, tile-size-and-SRAM-capacity-dependent IO-complexity proof is a separate, deeper technical contribution.

One-Line Takeaway

Takeaway: Tiled attention never materializes the full score matrix in HBM — small blocks stay in fast SRAM with an online softmax accumulator — reducing real HBM ↔ SRAM traffic without needing a single universal asymptotic formula to explain why.

Question 21: Given a tiny sequence length and head dimension, compute the simplified HBM-access-count comparison between naive and tiled attention, and explain why the reduction ratio grows with sequence length.

[ESSENTIAL]

Conversational Answer

"Module 04's own real, verified hand calc used a simplified accounting — explicitly labeled intuition-level, not the paper's exact proof — comparing naive attention's real HBM traffic (dominated by full-matrix materialization, scaling with N^2) against tiled attention's (scaling with $N \times d$). At a tiny $N=8$, $d=4$: naive comes out to 384 access units, tiled to 128 — a real 3.0x reduction. At $N=64$, same $d=4$: naive jumps to 17,408, tiled to only 1,024 — a real 17.0x reduction. Both figures were verified by direct code execution. The reduction ratio grows because naive traffic scales quadratically with sequence length while tiled traffic scales only linearly — so as sequences get longer, the gap between the two widens substantially, meaning FlashAttention's real practical benefit grows *more* pronounced at longer context, not less."

Intuitive Example

- Doubling the length of a shopping list roughly doubles how long it takes to write each item down once (linear), but if you also had to cross-reference every item against every other item on the list (quadratic), doubling the list length would roughly quadruple that cross-referencing work.

Key Interview Points

- **N=8, d=4**: real computed reduction ratio $\approx 3.0x$.
 - **N=64, d=4**: real computed reduction ratio $\approx 17.0x$ — grown substantially at 8x the sequence length.
 - **Why the gap widens**: naive traffic $\sim N^2$, tiled traffic $\sim N \cdot d$ — the quadratic-vs-linear gap grows with N .
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{HBM}_{\text{naive}} \approx 4N^2 + 4Nd \quad \text{HBM}_{\text{tiled}} \approx 4Nd$$

Explicitly a simplified accounting for intuition, not the paper's exact IO-complexity result.

Production Perspective & Trade-offs

This growing-benefit-at-longer-context pattern is a real, direct reason FlashAttention-class techniques matter increasingly for long-context production workloads, where naive attention's real quadratic HBM cost would otherwise dominate wall-clock latency even more severely.

Common Mistakes

1. Assuming FlashAttention's benefit is roughly constant across sequence lengths — the real, verified pattern is the opposite: the reduction ratio grows substantially with N .
2. Presenting this simplified formula as the paper's actual, exact IO-complexity result rather than an intuition-level approximation explicitly labeled as such.

COMMON FOLLOW-UP QUESTIONS

Q: Would this reduction ratio keep growing indefinitely at even longer real sequence lengths?

Qualitatively yes, under this simplified accounting — the quadratic-vs-linear gap widens without bound as N grows, though real production kernel behavior also depends on tile-size/SRAM-capacity factors this simplified formula doesn't capture.

Q: Why does head dimension d appear in the tiled term but not change the naive term's dominant behavior?

Naive traffic's dominant $4N^2$ term comes from the materialized score matrix, which doesn't depend on d directly at this level of accounting — while the Nd terms (present in both, but dominant only for tiled) come from moving $Q/K/V$ /output themselves, which do scale with d .

One-Line Takeaway

Takeaway: A real, verified simplified hand calc showed the naive-vs-tiled HBM-traffic reduction ratio grow from $\approx 3.0\times$ ($N=8$) to $\approx 17.0\times$ ($N=64$) — tiling's real benefit gets *more* pronounced at longer sequences, not less.

Question 22: Precisely distinguish what FlashAttention optimizes from what PagedAttention optimizes — why are they complementary rather than competing?

[ESSENTIAL]

Conversational Answer

"FlashAttention is a compute-kernel optimization — it reduces real HBM traffic *during the attention computation itself*, for a single forward pass. PagedAttention is a memory-*management* technique — it manages how the *resulting* KV cache gets allocated and reused *across* a whole serving system's many concurrent requests. They operate at genuinely different layers: one is about how attention math gets executed efficiently; the other is about how the memory attention produces gets stored and shared efficiently across a fleet of requests. A real production system uses both together — FlashAttention computing each forward pass efficiently, PagedAttention managing the resulting KV cache's memory efficiently across the whole serving system — not as alternatives to choose between."

Intuitive Example

- FlashAttention is like an efficient assembly-line process for building one product quickly with minimal wasted motion; PagedAttention is like an efficient warehouse-shelving system for storing many finished products from many production runs — genuinely different problems, both worth solving, neither substituting for the other.

Key Interview Points

- **FlashAttention:** compute-kernel-level, reduces real HBM traffic within one attention computation.
 - **PagedAttention:** serving-system-level, manages real KV-cache memory allocation across many concurrent requests.
 - **Complementary, not competing:** different layers of the real serving stack, used together in production.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a layer-of-the-stack distinction, directly building on Question 13 and Question 19's individual definitions.

Production Perspective & Trade-offs

Real production serving engines (vLLM, TensorRT-LLM) genuinely combine both — FlashAttention-class kernels for the compute, PagedAttention-style memory management for serving — precisely

because they solve different, non-overlapping real problems.

Common Mistakes

1. Presenting FlashAttention and PagedAttention as alternative solutions to the same real problem — a common, real interview misconception this question directly targets.
2. Assuming using one makes the other unnecessary — a real production system benefits from both simultaneously, since they address genuinely separate bottlenecks.

COMMON FOLLOW-UP QUESTIONS

Q: Could a serving system use PagedAttention without FlashAttention, or vice versa?

Yes, technically — they're independent techniques operating at different layers, though a real production system typically wants both for maximum real benefit, since foregoing either leaves a real, separate inefficiency unaddressed.

Q: Which one would you prioritize implementing first in a resource-constrained real engineering effort?

Depends on the real, dominant bottleneck in the specific deployment — if real memory waste from naive allocation is capping concurrent capacity, PagedAttention's benefit may be more immediately impactful; if real per-request latency from attention computation itself dominates, FlashAttention-class kernels may matter more first.

One-Line Takeaway

Takeaway: FlashAttention optimizes real HBM traffic within the attention computation; PagedAttention optimizes real KV-cache memory management across serving requests — genuinely different layers, used together, not alternatives.

Question 23: Why can two attention implementations have identical FLOPs but meaningfully different real wall-clock latency?

[ESSENTIAL]

Conversational Answer

"Because real wall-clock time on a modern GPU is often dominated by memory-bandwidth-bound traffic, not by raw compute throughput — exactly Module 01's roofline framing. Two implementations that compute the exact same mathematical result, with the exact same FLOP count, can still take genuinely different real time if one of them moves substantially more data between slow HBM and fast SRAM than the other. FlashAttention versus naive attention is precisely this case: identical real math,

identical real FLOPs, but a real, substantial difference in memory traffic — and therefore a real, substantial difference in measured wall-clock latency."

Intuitive Example

- Two chefs following the exact same recipe with the exact same number of cooking steps can still finish at very different real times if one of them has to walk to a distant pantry for every single ingredient while the other keeps everything within arm's reach.

Key Interview Points

- **Same FLOPs, different latency:** real wall-clock time depends on memory traffic too, not FLOPs alone.
 - **Direct link to Module 01's roofline framing:** a memory-bandwidth-bound operation's real latency is governed by data movement, not compute throughput.
 - **FlashAttention vs. naive:** the canonical real example — identical math, different real IO, different real latency.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Directly follows from Question 2's roofline framing — an operation's real latency, for a memory-bandwidth-bound operation, is governed by $\text{Bytes moved} / \text{Bandwidth}$, independent of its FLOP count.

Production Perspective & Trade-offs

This is a real, general reason FLOP-count alone is an unreliable proxy for real production latency — profiling real memory traffic (or at minimum, measuring real wall-clock latency directly) is necessary to actually understand a workload's real performance characteristics.

Common Mistakes

1. Using FLOP count as the sole metric when comparing two implementations' expected real performance — a real, common oversimplification this question directly corrects.
2. Assuming this effect only applies to attention specifically — it's a general consequence of the roofline model, applicable to any memory-bandwidth-bound operation.

COMMON FOLLOW-UP QUESTIONS

Q: How would you empirically confirm that two implementations' latency difference is really due to memory traffic, not something else?

Real, direct measurement — comparing wall-clock latency and peak memory usage empirically (as Notebook 03 did), and ideally profiling real HBM traffic with GPU-level tools, rather than inferring it purely from FLOP counts.

Q: Does this mean FLOP count is a useless metric?

No — it remains the real, correct metric for compute-bound operations; the point is that it's an incomplete predictor of real latency specifically for memory-bandwidth-bound operations, where data movement dominates instead.

One-Line Takeaway

Takeaway: Identical FLOPs can still mean genuinely different real wall-clock latency when memory traffic differs — exactly why FlashAttention beats naive attention in practice despite computing the same real math.

Question 24: A real notebook found FlashAttention itself genuinely unavailable on the installed hardware/build, and compared a different tiled kernel (EFFICIENT_ATTENTION) against the naive MATH backend instead — with the real latency gap growing from $\approx 5x$ to $\approx 33x$ as sequence length grew 512→4096. Why does reporting the unavailability honestly matter more than silently substituting a workaround?

[ESSENTIAL]

Conversational Answer

"Notebook 03 checked all three real SDPA backends before running any timed comparison, and found FlashAttention genuinely unavailable — the exact real error was 'Torch was not compiled with flash attention,' a genuine build-level constraint on that specific hardware/software combination, separate from and in addition to a real GQA head-count kernel-compatibility issue found during earlier exploration. Rather than silently swapping in a different comparison and calling it 'FlashAttention,' the notebook explicitly reported the real constraint, then compared two backends that *were* genuinely available: EFFICIENT_ATTENTION (a real, conceptually similar tiled/memory-efficient kernel) against MATH (the naive backend). The real result was still strong — latency gap growing from $\approx 5.34x$ at 512 tokens to $\approx 33.07x$ at 4096 tokens, closely tracking the expected quadratic-vs-linear memory-scaling pattern. Reporting the unavailability honestly matters because silently substituting and mislabeling the

result would misrepresent what was actually measured — a different real kernel, not literally FlashAttention itself."

Intuitive Example

- If a specific brand of tool isn't available for a real product test, honestly noting that and testing a comparable alternative tool instead — rather than testing the alternative but reporting it under the original brand's name.

Key Interview Points

- **Real constraint found:** FlashAttention genuinely unavailable on this exact build ("not compiled with flash attention").
 - **Honest fallback applied:** compared EFFICIENT_ATTENTION vs. MATH instead, explicitly reported as such.
 - **Real result:** latency gap grew from $\approx 5.34x$ to $\approx 33.07x$ across 512→4096 tokens — still a strong, genuine finding.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this question is about honest reporting discipline applied to Question 21's real underlying IO-aware mechanism.

Production Perspective & Trade-offs

This is a real, general engineering discipline: when a planned real tool or configuration isn't available, report that constraint explicitly and adapt rather than quietly substituting something else under the original label — misattributed results erode the real value of empirical verification.

Common Mistakes

1. Describing this notebook's real result as "we measured FlashAttention's speedup" — the actual measured backend was EFFICIENT_ATTENTION, a real, distinct (though conceptually related) kernel.
2. Assuming an unavailable planned experiment should simply be skipped rather than honestly reported with a genuine, clearly-labeled alternative substituted.

COMMON FOLLOW-UP QUESTIONS

Q: What did the real per-doubling memory-scaling pattern show?

MATH's real peak memory scaled by roughly 3.0x-3.8x per sequence-length doubling (approaching the expected quadratic pattern), while EFFICIENT_ATTENTION's scaled by only roughly 1.25x-1.57x (much closer to linear) — a real, measured pattern consistent with (though not a direct measurement of) the naive-vs-tiled HBM-traffic distinction.

Q: Does this real result still support Module 04's general claims despite not literally testing FlashAttention?

Yes — EFFICIENT_ATTENTION is a real, conceptually similar tiled/IO-aware kernel, so the real observed latency/memory pattern is consistent with the same underlying mechanism Module 04 describes, honestly reported as measuring that specific kernel rather than FlashAttention by name.

One-Line Takeaway

Takeaway: A real, honestly-reported hardware constraint (FlashAttention unavailable) led to a genuine, still-strong $\approx 5x \rightarrow \approx 33x$ real latency-gap finding using an honestly-labeled alternative kernel — reporting the constraint mattered more than silently mislabeling a substitute.

5. Quantization for Inference (Q25–Q30)

Question 25: Name the three distinct real quantization targets this topic covers — weights, activations, and KV cache — and explain why each has its own real accuracy/speed/memory trade-off profile.

[ESSENTIAL]

Conversational Answer

"Weight quantization targets the model's real, fixed parameters — typically quantized once, offline, and generally the most forgiving target since weights are static and can be calibrated carefully in advance. Activation quantization targets real intermediate values computed on the fly during the forward pass — a harder real accuracy trade-off, since activations vary per-input and can't be calibrated the same static way. KV-cache quantization targets the real, per-token key/value vectors accumulated across a whole generation — a direct lever specifically valuable in the long-context/high-concurrency regime where Module 02 already established KV cache can dominate real memory, but genuinely riskier for accuracy since quantization errors here can compound across a long real sequence."

Intuitive Example

- Rounding a fixed blueprint's measurements once, carefully, ahead of time (weights) is safer than rounding live, in-the-moment sensor readings on the fly (activations) — and rounding a running tally that keeps accumulating over a long process (KV cache) risks compounding small errors the longest.

Key Interview Points

- **Weights:** fixed, calibrated offline — generally the most forgiving real target.
 - **Activations:** computed on the fly per input — harder real accuracy trade-off.
 - **KV cache:** accumulated across a whole generation — genuine risk of compounding errors over a long real sequence.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Mem} = N_{\text{params}} \times \frac{\text{bits}}{8}$$

applies identically to all three targets — the real distinction between them is accuracy risk and calibration difficulty, not the memory-footprint formula itself.

Production Perspective & Trade-offs

Real production deployments often quantize these three targets at different real precisions independently — e.g., aggressive weight quantization (INT4) alongside more conservative KV-cache precision — precisely because their real accuracy-risk profiles differ.

Common Mistakes

1. Treating "quantization" as a single, undifferentiated lever rather than three real, distinct targets with different real risk profiles.
2. Assuming KV-cache quantization is exactly as safe as weight quantization simply because the memory formula looks the same — the real accuracy risk mechanism is genuinely different (Question 5 partially echoes this from Module 02's own KV-cache-specific framing).

COMMON FOLLOW-UP QUESTIONS

Q: Which target would you quantize most aggressively in a real, accuracy-sensitive production deployment?

Typically weights first — the most forgiving real target with the most mature tooling (GPTQ, AWQ) — before considering aggressive activation or KV-cache quantization, which carry real, higher accuracy risk.

Q: Why does KV-cache quantization specifically matter for long-context serving?

Because Module 02 already established KV cache can become the real dominant memory consumer at long context/high concurrency — quantizing it is a real, direct lever on exactly that dominant cost, not a marginal optimization.

One-Line Takeaway

Takeaway: Weights, activations, and KV cache are three real, distinct quantization targets, each with a genuinely different accuracy/speed/memory trade-off profile — not one undifferentiated lever.

Question 26: Walk through the bytes-per-parameter formula — why does it apply identically to weights, activations, and KV cache?

[ESSENTIAL]

Conversational Answer

"The real memory a set of values consumes at a given precision is simply the count of those values times the real number of bytes each one occupies at that bit-width — that arithmetic is identical regardless of whether the values in question are model weights, activation tensors, or KV-cache entries. The formula doesn't care *what* the values represent, only *how many* there are and *how many bits* each one uses. What differs across the three targets isn't the formula — it's the real accuracy sensitivity and calibration difficulty of applying a given precision reduction to that specific kind of value (Question 25)."

Intuitive Example

- Whether you're rounding a list of prices, a list of temperatures, or a list of distances, the real storage-space savings from using fewer digits per number follows the identical arithmetic — the *meaning* of the numbers doesn't change how much space rounding them saves.

Key Interview Points

- **Formula is target-agnostic:** $\text{Mem} = N_{\text{params}} \times \text{bits}/8$ applies identically regardless of what the values represent.
 - **What actually differs across targets:** real accuracy sensitivity, not the memory arithmetic itself.
 - **Direct implication:** real memory savings from a given bit-width reduction are exactly predictable regardless of target.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Mem} = N_{\text{params}} \times \frac{\text{bits}}{8}$$

Production Perspective & Trade-offs

This target-agnostic formula is exactly why Module 05's real hand calc could compute weight memory and KV-cache memory reductions using the identical formula shape (Question 27) — the real savings arithmetic doesn't need to be re-derived per target.

Common Mistakes

1. Assuming a more complex, target-specific memory formula is needed for KV cache versus weights — the real underlying arithmetic is identical; only the real value count and its source differ.
2. Confusing "the formula is the same" with "the real risk/trade-off is the same" — the formula's universality doesn't imply the accuracy trade-off is equally safe across targets (Question 25).

COMMON FOLLOW-UP QUESTIONS

Q: Does this formula account for any real quantization overhead (e.g., scale/zero-point storage)?

No — this is the simplified core formula for the quantized values themselves; real production quantization schemes add a real, typically small overhead for storing per-group scale factors, not captured in this base formula.

Q: How would you verify this formula's real predictions match actual measured memory?

Directly, as Notebook 04 did — loading a real model at each target precision and reading its actual real memory footprint, then comparing against the formula's predicted value.

One-Line Takeaway

Takeaway: The bytes-per-parameter formula is identical across weights, activations, and KV cache — what differs across targets is real accuracy risk, not the underlying memory arithmetic.

Question 27: Given a parameter count and a KV-cache configuration, compute real memory footprint at FP16, INT8, and INT4 for both targets.

[ESSENTIAL]

Conversational Answer

"Module 05's own real, verified hand calc: for an illustrative 7-billion-parameter model, weight memory comes out to 13.04GB at FP16, 6.52GB at INT8, and 3.26GB at INT4 — each halving of bit-width exactly halves real memory, verified by direct code execution. Reusing Module 02's own high-concurrency KV-cache scenario (batch 32, sequence length 4096, MHA), KV-cache memory comes out to 64.00GB at FP16, 32.00GB at INT8, and 16.00GB at INT4 — the same exact halving pattern. Combined, total real memory at this workload drops from 77.04GB (FP16) to just 19.26GB (INT4) — a real, direct 4x total reduction, verified exactly against the code's own assertions."

Intuitive Example

- Switching a large document from full-resolution scans to progressively more compressed image formats — each step down in quality roughly halves the real file size, in a predictable, exactly-computable way.

Key Interview Points

- **Weights (7B model):** 13.04GB (FP16) → 6.52GB (INT8) → 3.26GB (INT4) — exact halving each step.
 - **KV cache (Module 02's scenario):** 64.00GB (FP16) → 32.00GB (INT8) → 16.00GB (INT4) — same exact pattern.
 - **Combined total:** 77.04GB (FP16) → 19.26GB (INT4) — a real, verified 4x total reduction.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same formula as Question 26, evaluated at three bit-widths for each target — the halving pattern is exact and predictable at every step, verified via direct assertion in Module 05's reference code.

Production Perspective & Trade-offs

This real, computed 4x total memory reduction (weights + KV cache combined) is directly reusable capacity in production — freeing that memory for higher real concurrency or longer real context within the same fixed GPU budget.

Common Mistakes

1. Computing weight and KV-cache memory reduction independently without checking they use consistent units (binary GB throughout, matching Module 02's established convention) — a real, previously-caught unit-mismatch bug (Question 9) underscores why this matters.
2. Assuming this real 4x total-memory reduction automatically implies a proportional real latency reduction — that's a separate, distinct question (Question 28/30) the formula alone doesn't answer.

COMMON FOLLOW-UP QUESTIONS

Q: Why does going from FP16 to INT8 to INT4 always exactly halve memory at each step?

Because bit-width itself is exactly halved at each step (16→8→4), and the formula is linear in bits — so the real memory reduction ratio always exactly matches the bit-width reduction ratio.

Q: Does this 4x combined reduction hold at a different real workload (different batch size/sequence length)?

The real *ratio* (4x from FP16 to INT4) holds regardless of workload, since it's driven purely by the bit-width reduction; the real absolute GB figures would differ at a different batch size or sequence length, per Module 02's own workload-dependent framing.

One-Line Takeaway

Takeaway: A real, verified hand calc showed weights and KV cache both drop exactly with bit-width — combined real memory fell from 77.04GB (FP16) to 19.26GB (INT4), a genuine, code-verified 4x total reduction.

Question 28: Why does a real memory-footprint reduction from quantization not guarantee a proportional real latency reduction? Name the two separate real conditions that determine whether it does.

[ESSENTIAL]

Conversational Answer

"The real memory-footprint reduction from quantization is direct and guaranteed by the arithmetic — that part always holds. Whether it translates into a real *speed* improvement depends on two genuinely separate real conditions. First: is the workload actually memory-bandwidth-bound in the first place, per Module 01's roofline framing? A compute-bound workload sees little real speed benefit from smaller values, since compute time — not data movement — was the real constraint to begin with. Second: does the serving hardware and kernel implementation genuinely support fast native execution at that specific lower precision? Without real native low-precision kernel support, values may need to be

upcast back to a higher precision before computing, adding real overhead that can partially or fully offset the memory win — or, as this topic's own real notebook found, overwhelm it entirely."

Intuitive Example

- Switching to a lighter suitcase doesn't necessarily make your trip faster if the slow part was actually waiting in a security line, not carrying the bag — and if the lighter suitcase requires an awkward extra repacking step at every checkpoint, it could even slow you down.

Key Interview Points

- **Memory reduction:** always real and guaranteed by the formula.
 - **Condition 1:** whether the workload is genuinely memory-bandwidth-bound (Module 01's roofline framing) — otherwise smaller values don't help much.
 - **Condition 2:** whether the hardware/kernel genuinely supports fast native low-precision execution — otherwise real dequantization overhead can offset or exceed the win.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this question connects Module 05's caveat directly back to Module 01's roofline framing (Question 2).

Production Perspective & Trade-offs

This is exactly why quantization decisions in production need real, empirical latency validation on the actual target hardware/kernel stack — the memory-footprint formula alone (Question 27) cannot predict the real latency outcome.

Common Mistakes

1. Assuming a real memory-footprint win from quantization automatically implies a proportional real speed win — the single most common, real overclaim this topic's own notebook directly disproved.
2. Skipping real hardware/kernel-specific latency validation because the memory-savings math "looks obviously good" on paper.

COMMON FOLLOW-UP QUESTIONS

Q: How would you determine, before deploying, whether a specific real workload is likely to see a genuine latency win from quantization?

Check its real roofline position (Module 01) and confirm the target hardware/kernel stack has genuine native support for the target precision — both real, empirically-checkable conditions, not assumptions.

Q: Is this caveat specific to a particular quantization library, or more general?

General — it's a structural consequence of how quantized compute actually executes on real hardware, though the specific real magnitude of any overhead is genuinely library/kernel/hardware-specific (Question 30's concrete example).

One-Line Takeaway

Takeaway: A real quantization memory win requires two separate real conditions to also produce a speed win — genuine memory-bandwidth-bound-ness and genuine native low-precision kernel support — neither guaranteed by the memory formula alone.

Question 29: Why might KV-cache quantization carry more real accuracy risk than weight quantization?

[ESSENTIAL]

Conversational Answer

"Weight values are fixed, offline parameters — they can be calibrated carefully, once, against a representative real calibration dataset before deployment. KV-cache values are real, per-input activations, freshly computed and accumulated across an entire live generation — there's no equivalent offline calibration opportunity, since each real sequence's KV-cache content is genuinely different. Worse, quantization errors introduced into the KV cache at an early real token can compound as that cache gets read again and again across every subsequent decode step of a long real sequence — a static weight-quantization error, by contrast, doesn't compound the same way across a generation, since it's the identical fixed error applied consistently."

Intuitive Example

- A precisely-calibrated fixed measuring instrument (weights) stays accurate throughout a whole project, but small rounding errors introduced into a running, cumulative tally (KV cache) can compound and drift further off as the tally grows longer.

Key Interview Points

- **Weights:** fixed, offline-calibratable — no compounding across a generation.
 - **KV cache:** real, per-input, accumulated live — no equivalent offline calibration, and errors can compound across a long sequence.
 - **Real practical implication:** KV-cache quantization generally warrants more real, careful accuracy validation than weight quantization.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a real accuracy-risk mechanism distinction, not a quantitative claim.

Production Perspective & Trade-offs

Real production deployments often apply more conservative real precision to KV cache than to weights precisely for this reason — even when the memory-savings arithmetic (Question 26) looks identical on paper.

Common Mistakes

1. Applying the same aggressive real quantization precision to KV cache as to weights, based purely on the memory-formula symmetry, without separately validating real accuracy impact.
2. Assuming KV-cache quantization errors are bounded/local — the real risk is specifically that they can compound across a long generation, not stay isolated to one token.

COMMON FOLLOW-UP QUESTIONS

Q: How would a real production team validate KV-cache quantization's accuracy impact specifically?

Real, task-specific evaluation on long-generation benchmarks where compounding effects would actually manifest — not just a single-token or short-sequence accuracy check that wouldn't reveal compounding.

Q: Does this compounding risk apply equally to weight quantization if a model does many sequential inference calls?

No — each individual inference call starts fresh from the same fixed, identically-quantized weights; there's no real cross-call accumulation the way there is within a single sequence's own growing KV cache.

One-Line Takeaway

Takeaway: KV-cache quantization lacks weight quantization's offline-calibration opportunity and risks compounding errors across a long real generation — a genuinely higher real accuracy risk than static weight quantization.

Question 30: A real notebook measured genuine 36.2% (INT8) and 54.3% (INT4) memory reductions on a small model/consumer GPU, but found generation became slower, not faster — INT8 by 3.37x, INT4 by 1.34x. What real, plausible mechanism explains a memory win producing a net latency loss?

[ESSENTIAL]

Conversational Answer

"Notebook 04 measured this directly and honestly, reporting exactly what happened rather than adjusting the finding to match expectations: FP16 was the real *fastest* configuration at 97.009ms/token; INT8, despite a genuine 36.2% memory reduction, measured 327.145ms/token — a real 3.37x slowdown; INT4, despite a genuine 54.3% memory reduction, measured 130.079ms/token — a real 1.34x slowdown. The real, plausible mechanism, consistent with Question 28's framing: the quantization library's INT8/INT4 kernels perform real dequantization work on every matmul call, and at this small model size (0.5B parameters) and batch size (1) on a consumer GPU, that real per-call dequantization overhead outweighed the real memory-bandwidth savings — this workload was never memory-bandwidth-bound enough, at this small scale, for the smaller real footprint to translate into a real speed win."

Intuitive Example

- Switching to a lighter, foldable suitcase that requires an extra 30-second unfolding step at every single checkpoint on a trip with many checkpoints — the total added unfolding time can exceed whatever time the lighter weight itself saved.

Key Interview Points

- **Real result:** FP16 fastest (97.009ms/token); INT8 3.37x slower; INT4 1.34x slower — despite genuine memory savings (36.2%/54.3%).
 - **Plausible real mechanism:** per-call dequantization overhead, at this small model/batch-size/consumer-GPU combination, outweighed the real bandwidth savings.
 - **Directly confirms Question 28:** neither real condition (memory-bandwidth-bound workload, native fast low-precision kernel support) held cleanly here.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this is Question 28's caveat demonstrated with real, measured numbers, on this specific real hardware/model/batch-size combination.

Production Perspective & Trade-offs

This is a real, concrete, cautionary case study for any team considering quantization purely from a memory-savings projection — genuine empirical latency validation on the *actual* target deployment scale (model size, batch size, hardware) is necessary, since this result would not have been predictable from the memory formula alone.

Common Mistakes

1. Generalizing this specific real result ("quantization always makes things slower") beyond its actual scope — it's a genuine finding for this specific small model, batch size 1, and consumer GPU combination, not a universal claim about quantization.
2. Dismissing a real, honestly-reported negative result as "something went wrong" rather than treating it as exactly the kind of genuine finding worth reporting as-is.

COMMON FOLLOW-UP QUESTIONS

Q: Why might INT8 have been slower than INT4 in this specific real result — isn't INT8 usually considered "safer" or more optimized?

A real, honest, specific data point rather than a general rule — plausibly reflecting this particular quantization library's INT8 kernel path being less optimized for this small-model/single-sequence regime than its INT4 path on this exact hardware.

Q: Would this same result likely hold at a much larger batch size or model size?

Not necessarily — larger batch sizes shift the workload's real roofline position (Question 4's batching-shifts-intensity mechanism), which could change whether the memory-bandwidth savings actually dominate; this specific result is scoped to the tested configuration, not asserted more broadly.

One-Line Takeaway

Takeaway: A real notebook found quantization's genuine memory savings (36.2%/54.3%) came with a genuine net latency *loss* (3.37x/1.34x slower) on this specific small-model/consumer-GPU setup — a strong, honest, real confirmation that memory wins don't guarantee speed wins.

6. Batching Strategies — Static, Dynamic & Continuous Batching (Q31–Q36)

Question 31: Why does static batching waste real GPU compute even when the hardware itself has spare capacity?

[ESSENTIAL]

Conversational Answer

"Static batching launches a fixed group of sequences together and doesn't let any of them finish, or free their slot, until the whole batch's *slowest* member is done. Real sequences generate different numbers of tokens — some finish in a handful of steps, others run much longer. Every sequence that finishes early just sits there, its slot real, allocated, and idle, for however much longer the slowest sequence takes. The hardware has real spare compute capacity sitting right there — the GPU isn't fully utilized — but static batching's rigid all-finish-together structure prevents that spare capacity from being handed to a new, real waiting request until the entire batch completes."

Intuitive Example

- A tour bus that won't let any passenger off, or pick up a new one, until every single passenger's own separate destination has been reached — even the passengers who arrived at their stop miles ago just sit there, seat occupied, going nowhere useful.

Key Interview Points

- **Static batching's rigid constraint:** no slot frees until the whole batch's slowest member finishes.
 - **Real waste source:** early-finishing sequences' slots sit idle rather than being reused.
 - **Root cause:** real sequence-length variance combined with the fixed-group structure, not a hardware limitation.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — the quantitative version of this waste is Question 33's worked example.

Production Perspective & Trade-offs

This is the direct real motivation for continuous batching (Question 32) — removing the fixed-group constraint so idle slots get reused immediately rather than sitting empty until the whole batch finishes.

Common Mistakes

1. Assuming static batching's inefficiency is a hardware limitation — it's a real scheduling/structural limitation, not a raw compute-capacity problem.
2. Overlooking that this waste grows worse the more real sequence-length variance a workload has — a workload with uniform real lengths wouldn't suffer nearly as much from static batching's constraint.

COMMON FOLLOW-UP QUESTIONS

Q: Would static batching still waste compute if every real sequence in a batch happened to be exactly the same length?

Much less so — the real waste specifically comes from length **variance** within a batch; uniform-length sequences would all finish together, largely avoiding this particular inefficiency.

Q: Is static batching ever still a reasonable real choice?

In workloads with genuinely low real length variance or where implementation simplicity matters more than maximizing throughput, static batching's real simplicity can be an acceptable trade-off — though most production serving engines have moved to continuous batching specifically to avoid this waste.

One-Line Takeaway

Takeaway: Static batching wastes real GPU compute because it won't free a finished sequence's slot until the whole batch's slowest member completes — a structural, not hardware, limitation.

Question 32: Walk through how continuous batching removes the fixed-group constraint — what specifically happens the moment one sequence finishes?

[ESSENTIAL]

Conversational Answer

"The instant a sequence in a continuously-batched system finishes, its now-free slot gets handed to a new, real waiting request immediately — the rest of the batch doesn't pause, doesn't wait, and isn't otherwise affected. There's no fixed group anymore in the static-batching sense; the batch composition is fluid, constantly refilled as slots free up. This is the real mechanism behind vLLM/TGI-class serving engines' throughput gains: GPU slots stay real, continuously occupied by genuinely useful work, rather than idling out waiting for a batch's slowest member."

Intuitive Example

- A bus that lets a passenger off the instant they reach their destination and immediately picks up the next real waiting passenger for that now-empty seat, without the rest of the bus's passengers needing to wait or be affected at all.

Key Interview Points

- **The moment a sequence finishes:** its slot is immediately backfilled with a new real waiting request.
 - **No fixed group:** batch composition is fluid, not locked to a static set launched together.
 - **Real mechanism behind throughput gains:** GPU slots stay continuously occupied by useful work.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — the quantitative benefit is Question 33's worked example.

Production Perspective & Trade-offs

This constant slot-freeing/refilling is exactly why continuous batching pairs naturally with PagedAttention (Question 17-adjacent) — KV-cache memory needs to be allocated and reclaimed just as constantly, and block-based allocation makes that cheap.

Common Mistakes

1. Describing continuous batching as "a bigger batch size" — it's a structural change to *when* slots get filled/freed, not simply a larger fixed group.
2. Assuming continuous batching requires no real scheduling logic — a real scheduler still has to decide which waiting request backfills which freed slot, and that policy matters (Question 34's worked example).

COMMON FOLLOW-UP QUESTIONS

Q: Does continuous batching change the real per-token compute cost of any individual sequence?

No — each sequence's own real per-token cost is unchanged; what changes is how efficiently the *batch as a whole* uses available GPU capacity across many sequences with different real lengths.

Q: What real system component decides which waiting request backfills a freed slot?

The serving engine's real scheduler — and the specific real policy it uses (e.g., strict arrival order vs. best-fit) directly affects how close real achieved utilization gets to the theoretical ceiling (Question 34).

One-Line Takeaway

Takeaway: Continuous batching immediately backfills a freed slot with a new real waiting request the instant a sequence finishes — no fixed group, no waiting for the batch's slowest member.

Question 33: Given a set of real sequences with different lengths and a real backfill queue, compute the real idle-compute waste under static vs. continuous batching.

[ESSENTIAL]

Conversational Answer

"Module 06's own real, verified worked example: four initial sequences needing $\{50, 200, 80, 400\}$ decode steps, batch width 4, so total real slot-capacity in the 400-step window is 1,600. Under static batching, used steps total 730 against 1,600 capacity — 870 wasted, about 54.4%. Under continuous batching, with a real backfill queue of $\{200, 100, 40, 10, 300, 200\}$ available to fill freed slots as they occur: used steps rise to 1,380 — only 220 wasted, about 13.75%. That's a real, computed 4.0x reduction in wasted compute. But it's not zero waste — one slot's real backfill candidate (300) didn't fit its remaining window, leaving a genuine, honest gap rather than a perfectly idealized 100% utilization result."

Intuitive Example

- A restaurant that seats new diners the instant a table frees up (continuous) serves dramatically more real meals in an evening than one that only reseats every table at once, but even the continuous approach can occasionally leave a table briefly empty if the next party waiting doesn't quite fit the remaining time slot.

Key Interview Points

- **Static:** 730/1,600 used → 870 wasted ($\approx 54.4\%$).
- **Continuous:** 1,380/1,600 used → 220 wasted ($\approx 13.75\%$) — a real 4.0x reduction.
- **Honest, non-idealized result:** one real backfill candidate genuinely didn't fit, leaving nonzero waste even under continuous batching.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — a real, simulated greedy backfill algorithm (each freed slot filled by the next queued request that fits its remaining window) computed these figures directly, verified by code execution.

Production Perspective & Trade-offs

The nonzero real waste under continuous batching in this exact example is itself a useful, honest data point: real achieved utilization depends on real scheduling policy quality and the real mix of waiting request lengths, not on continuous batching as a technique alone guaranteeing near-100% utilization.

Common Mistakes

1. Treating continuous batching's real benefit as "eliminates waste" rather than "substantially reduces waste" — this exact worked example's own nonzero 13.75% result directly contradicts a zero-waste claim.
2. Assuming the specific 4.0x reduction figure generalizes to every workload — it's a result of this exact worked example's real length distribution and backfill queue.

COMMON FOLLOW-UP QUESTIONS

Q: Why did one real backfill candidate specifically fail to fit?

The greedy scheduling policy used (strict-order-fit) placed a length-300 candidate against a slot with only 200 real steps remaining in the window — it didn't fit, and the greedy algorithm didn't look ahead to try a better-fitting later candidate instead, an honest limitation of that specific real scheduling policy.

Q: Would a smarter real scheduling policy have achieved less waste in this same example?

Plausibly — a look-ahead or best-fit policy might have matched the length-300 candidate to a different, better-fitting slot, though this worked example specifically used simple strict-order-fit scheduling and reported its real, honest result rather than optimizing the scheduler to produce a cleaner number.

One-Line Takeaway

Takeaway: A real, verified worked example showed continuous batching cutting idle waste from $\approx 54.4\%$ to $\approx 13.75\%$ (a real 4.0x reduction) — genuine and substantial, but honestly nonzero, not an idealized 100%-utilization result.

Question 34: Why is "continuous batching always improves both throughput and latency" an overclaim? What real, concrete gap can remain even under correct continuous-batching scheduling?

[ESSENTIAL]

Conversational Answer

"Continuous batching generally improves real throughput and GPU utilization — that part holds up consistently. But real latency for any *specific* request depends on the real mix and order of other requests sharing the system at that moment, and that's genuinely scheduler- and workload-dependent, not an automatic guarantee. Question 33's own worked example makes this concrete: even under correct, real continuous-batching logic, one request's real backfill candidate simply didn't fit its available slot — a genuine, remaining scheduling gap. A request stuck behind others in a real, unfavorable ordering can see *worse* real latency than it might have under a different scheduling order, even though the system's overall real throughput improved."

Intuitive Example

- A hospital's overall real patient-throughput can improve with a smarter triage system, while a specific individual patient's own real wait time can still occasionally be worse than under a simpler system, depending on exactly who else happened to be waiting at the same time.

Key Interview Points

- **Throughput claim:** generally holds — continuous batching does improve real GPU utilization and aggregate throughput.
 - **Latency claim:** genuinely workload/scheduler-dependent, not automatic for every individual request.
 - **Concrete real gap:** Question 33's own worked example left a real, honest nonzero-waste gap even under correct scheduling logic.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this directly interprets Question 33's real result as a general caveat.

Production Perspective & Trade-offs

Real production teams should track per-request latency distributions (not just aggregate throughput) when evaluating continuous-batching performance, since aggregate throughput improving doesn't guarantee every individual request's real experience improved too.

Common Mistakes

1. Reciting "continuous batching always improves throughput and latency" as an unconditional fact — a real, common overclaim this topic explicitly qualifies.
2. Only monitoring aggregate throughput in production without also tracking real per-request latency variance, which can hide genuine scheduling-order-dependent regressions for specific requests.

COMMON FOLLOW-UP QUESTIONS

Q: How would you detect a real scheduling-order-dependent latency regression in production?

Monitoring real per-request latency distribution (not just aggregate/average), watching for requests that wait disproportionately long relative to their own real length — a signal the scheduling policy is leaving some requests genuinely stuck.

Q: Does this mean continuous batching is a worse choice than static batching for latency-sensitive workloads?

Not generally — continuous batching's real throughput and typical-case latency benefits usually still dominate; the caveat is that it's not an unconditional guarantee for every individual request, not that static batching is typically the better choice.

One-Line Takeaway

Takeaway: Continuous batching's throughput benefit is real and consistent, but its latency benefit is genuinely workload/scheduler-dependent — Module 06's own worked example left a real, honest gap even under correct scheduling logic.

Question 35: How does continuous batching's constant slot-freeing/refilling directly motivate pairing it with PagedAttention in production serving engines?

[ESSENTIAL]

Conversational Answer

"Continuous batching frees and refills slots constantly — which means the KV-cache memory associated with each slot needs to be allocated and reclaimed just as constantly, not once per static batch launch. Under contiguous allocation, that would mean freeing and re-reserving a full worst-case-sized memory region every single time a slot turns over — real, expensive, frequent overhead. PagedAttention's block-based allocation makes that constant allocation/reclamation cheap and low-fragmentation instead, since freeing a sequence just returns its real, small blocks to the shared pool rather than releasing one giant contiguous region. The two techniques target genuinely different real

problems (Question 22), but continuous batching's operational pattern specifically demands exactly the kind of cheap, frequent (re)allocation PagedAttention provides."

Intuitive Example

- A hotel with guests checking in and out constantly needs an efficient room-turnover system (quick cleaning/reassignment of individual rooms) far more than a hotel where every guest arrives and leaves on the same fixed schedule — the *frequency* of turnover is what makes an efficient reallocation mechanism matter.

Key Interview Points

- **Continuous batching's operational demand:** constant, frequent slot freeing and refilling.
 - **Contiguous allocation's mismatch:** expensive to free/re-reserve a full worst-case region on every turnover.
 - **PagedAttention's fit:** cheap, low-fragmentation (re)allocation via returning small blocks to a shared pool.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is an architectural-compatibility argument connecting Question 13's PagedAttention mechanism to Question 32's continuous-batching mechanism.

Production Perspective & Trade-offs

Real production serving engines (vLLM being the canonical real example) implement both together specifically for this reason — the pairing isn't incidental, it's a direct, real operational fit between the two techniques' respective demands and capabilities.

Common Mistakes

1. Treating PagedAttention and continuous batching as independent features that merely happen to be bundled together in real engines, rather than recognizing the real, direct operational reason they pair naturally.
2. Assuming continuous batching could work equally well with contiguous allocation given "enough engineering effort" — the real, frequent reallocation cost under contiguous allocation is a structural mismatch, not just an implementation inconvenience.

COMMON FOLLOW-UP QUESTIONS

Q: Could continuous batching function at all without PagedAttention-style allocation?

Technically yes, but with real, substantial reallocation overhead on every slot turnover under contiguous allocation — functionally viable but real, measurably less efficient than the paged pairing.

Q: Does PagedAttention require continuous batching to be useful?

No — PagedAttention's allocation-efficiency benefit (Question 15) is real and standalone even under static batching, though its full value is more fully exploited when paired with continuous batching's frequent reallocation pattern.

One-Line Takeaway

Takeaway: Continuous batching's constant slot turnover demands cheap, frequent KV-cache (re)allocation — exactly what PagedAttention's block-based design provides, which is why production engines pair the two.

Question 36: A real notebook's batching experiment showed throughput scaling well from batch size 1 to 4, then real per-request latency jumping 278% at batch size 8 — caused by two real sequences that never emitted a natural stop token. What production lesson does this straggler event illustrate that a synthetic example might not?

[ESSENTIAL]

Conversational Answer

"Notebook 05 measured this directly, without staging it: real throughput scaled from 5.25 tok/s at batch size 1 up to 33.41 tok/s at batch size 4 — a genuine 6.36x improvement. At batch size 8, though, real throughput actually dipped slightly to 32.71 tok/s, and real per-request latency jumped 278%, from 202.0ms to 764.2ms. The cause was visible directly in the real per-request generation lengths: [8, 10, 2, 7, 11, 80, 80, 2] — two of the eight real sequences ran the entire 80-token budget without ever emitting a real stop token, a genuine 40x spread between the shortest and longest real generation. Because this was static (non-continuous) batching, those two real stragglers forced the *entire* batch's wall-clock time to stretch to accommodate them. What makes this more valuable than a synthetic example is that it wasn't constructed to prove a point — it emerged naturally from real model behavior on a real, ordinary prompt set, exactly the kind of straggler risk a production system has to plan for even when nobody deliberately engineered it."

Intuitive Example

- A carpool that arrives late every day not because of a staged worst-case scenario, but because, purely by chance, two of the eight real riders that particular week happened to have unusually long real errands before pickup — the lesson about carpool design risk is more convincing precisely because nobody planned it that way.

Key Interview Points

- **Real result:** throughput scaled 6.36x from batch 1→4, then dipped slightly at batch 8 with a real 278% latency jump.
 - **Real cause:** two of eight real sequences never emitted a stop token, running the full 80-token budget (a real 40x length spread).
 - **Why it's more valuable than synthetic:** it emerged naturally from real model behavior, not a constructed worst-case example.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No new formula — this is Question 31's static-batching-waste mechanism, observed live rather than only hand-computed.

Production Perspective & Trade-offs

This real, unstaged event is a genuine, concrete argument for continuous batching (Question 32) in any real production deployment where request-length variance is expected — which, per this exact real result, can arise even from an ordinary, non-adversarial prompt mix.

Common Mistakes

1. Assuming straggler risk only matters for adversarially long real prompts — this result shows it can arise from ordinary real generation behavior (a model simply not emitting a stop token within budget) without any adversarial intent.
2. Treating a single real batch-size-8 measurement as conclusive about typical production behavior — it's one genuine, real data point illustrating a real risk class, not a comprehensive benchmark.

COMMON FOLLOW-UP QUESTIONS

Q: Why might a real sequence fail to emit a stop token within budget?

A real, model-specific behavior — the model's own generation may simply not naturally conclude within the given token budget for that specific real prompt, independent of any adversarial input.

Q: How does this real straggler event connect to Question 18's grounded PagedAttention simulation?

Directly — the same real, straggler-containing generation lengths from this exact experiment were fed into that simulation as its real input data, carrying this genuine finding forward into a second, related analysis.

One-Line Takeaway

Takeaway: An unstaged, real straggler event — two of eight sequences running a full token budget with no natural stop — caused a real 278% latency spike at batch size 8, a genuine, naturally-occurring illustration of static batching's core risk.

7. Speculative Decoding & Decoding-Time Optimizations (Q37–Q42)

Question 37: Walk through the draft-then-verify mechanism — why can the target model verify k candidate tokens in a single parallel forward pass, making verification far cheaper than generating those same k tokens one at a time?

[ESSENTIAL]

Conversational Answer

"A small, fast draft model proposes k candidate tokens one at a time, autoregressively — that part is still sequential for the draft model. But verification is genuinely different: the target model can check all k candidate positions *simultaneously* in one forward pass, because verification just means computing what the target model *would have* predicted at each of those positions given the tokens before it — and all those positions' inputs are already known upfront (the draft's proposed tokens), unlike real autoregressive generation where each next token is genuinely unknown until sampled. That's why one target forward pass can verify k tokens at once, whereas generating k tokens autoregressively would require k genuinely sequential target forward passes."

Intuitive Example

- Checking a full page of a student's already-written answers against an answer key in one pass, versus asking the student to write and grade each answer one at a time before moving to the next

— the checking is genuinely parallelizable because every answer already exists to check against.

Key Interview Points

- **Draft phase:** still sequential — the draft model proposes tokens one at a time.
 - **Verification phase:** genuinely parallel — all k candidate positions' inputs are already known, so one forward pass checks them all.
 - **Why this is cheaper:** real, direct amortization of one expensive target forward pass across potentially several accepted tokens.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula here — the expected-accepted-tokens formula is Question 38's focus.

Production Perspective & Trade-offs

This parallel-verification mechanism is exactly why speculative decoding specifically targets the real memory-bandwidth-bound decode phase (Question 40) — it reduces how many *expensive, full* target forward passes are needed per real output token, without changing what each individual pass costs.

Common Mistakes

1. Claiming verification costs "the same real memory traffic as generating one token" — the precise mechanism is that verification checks k positions in one parallel pass, not that the traffic itself is unchanged.
2. Assuming the draft model's own generation is also parallelized — only verification is; the draft model still generates its k candidates sequentially.

COMMON FOLLOW-UP QUESTIONS

Q: Does verification always produce the exact same output as standard autoregressive decoding?

Yes, for greedy (or a correctly-implemented sampling) verification scheme — speculative decoding is a real, exact-output-preserving technique, provably identical in distribution to standard decoding from the target model alone.

Q: What happens to the draft model's own proposed tokens after verification?

The real, longest matching prefix of accepted draft tokens is kept, plus one real "bonus"/correction token from the target model at the first rejection point (or after the last accepted token if all were accepted) — Question 42's implementation walks through this concretely.

One-Line Takeaway

Takeaway: The target model verifies all k draft candidates in one parallel forward pass because their inputs are already known upfront — genuinely cheaper than k sequential autoregressive passes would be.

Question 38: Derive the expected-accepted-tokens formula under the simplified constant- α acceptance model, and explain precisely what it does *not* capture about real speedup.

[ESSENTIAL]

Conversational Answer

"Under the simplified model, each of k draft tokens is treated as accepted independently with a real, constant probability α — a simplification, since real acceptance probability actually varies token-to-token depending on how well the draft matches the target's distribution at that specific point. Under this simplified model, the expected number of tokens produced per verification round — accepted draft tokens plus the one bonus/correction token always produced — comes out to $E[\text{accepted}] = (1 - \alpha^{k+1}) / (1 - \alpha)$. What this formula precisely does *not* capture: the real relative cost of a draft-model forward pass versus a verification forward pass. It tells you how many tokens you'd expect per round, not how much real time that round actually costs — that's a genuinely separate, second calculation (Question 39)."

Intuitive Example

- Knowing how many items on average you'll successfully check off a shopping list per store visit doesn't by itself tell you how long each visit actually takes — that depends on a separate, real fact about how fast you can walk the store.

Key Interview Points

- **Formula:** $E[\text{accepted}] = (1 - \alpha^{k+1}) / (1 - \alpha)$ under a simplified constant- α model.
- **What it captures:** expected accepted tokens per verification round.
- **What it does not capture:** real draft/verification relative cost — necessary for actual speedup, but a separate calculation.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$E[\text{accepted}] = \frac{1 - \alpha^{k+1}}{1 - \alpha}$$

Explicitly labeled as giving expected accepted tokens under this simplified model — not the complete expected speedup.

Production Perspective & Trade-offs

This formula/speedup separation is a real, general discipline: a quantity that sounds like it should directly imply a performance outcome (accepted tokens $\rightarrow$ speedup) often requires an additional, separate real cost assumption before that implication actually holds.

Common Mistakes

1. Treating $E[\text{accepted}]$ itself as "the speedup" — a real, common conflation this question and Question 42 directly correct.
2. Assuming real acceptance probability is genuinely constant across draft positions — it's an explicit simplification, not an empirical claim about real model behavior.

COMMON FOLLOW-UP QUESTIONS

Q: What would $E[\text{accepted}]$ equal at $\alpha = 1$ (perfect acceptance)?

$k + 1$ — every draft token accepted plus the bonus token, the real theoretical ceiling this simplified formula approaches as $\alpha \rightarrow 1$.

Q: Why include the "+1" bonus token in the formula at all?

Because a verification round always produces at least one new real token — either the target's own correct token at the first rejection point, or (if all k draft tokens were accepted) one additional bonus token from the target's own prediction after the last accepted position.

One-Line Takeaway

Takeaway: $E[\text{accepted}] = (1 - \alpha^{k+1})/(1 - \alpha)$ gives expected accepted tokens under a simplified constant- α model — it deliberately does not capture real draft/verification cost, which speedup requires as a separate calculation.

Question 39: Given a real acceptance rate and draft length, compute expected accepted tokens per round; then, given a separate real draft/verification cost assumption, compute expected speedup as a distinct second step.

[ESSENTIAL]

Conversational Answer

"Module 07's own real, verified two-step hand calc: at $\alpha = 0.8$, $k = 4$, Step 1 gives $E[\text{accepted}] = (1 - 0.8^5)/(1 - 0.8) \approx 3.362$ tokens per round. Step 2 — kept explicitly separate — requires a stated real cost assumption: say a draft-model forward pass costs 0.2x a full verification pass. One speculative round then costs $k \times 0.2 + 1 = 1.8$ (in verification-pass units), while producing the same 3.362 tokens via standard autoregressive decoding would cost 3.362 passes. That gives a real expected speedup of $3.362/1.8 \approx 1.87\times$ — genuinely positive, but well short of the naive $k+1 = 5\times$ ceiling a surface reading of the formula might suggest, precisely because the real draft-model cost isn't free."

Intuitive Example

- Knowing you'll successfully check off about 3.4 items per store visit on average is useful, but computing whether that visit was actually *worth it* requires separately knowing how much time the visit itself cost compared to shopping for those same items one trip at a time.

Key Interview Points

- **Step 1 (real, verified):** $\alpha=0.8, k=4 \rightarrow E[\text{accepted}] \approx 3.362$ tokens/round.
 - **Step 2 (separate, requires a stated cost assumption):** draft cost = $0.2\times$ verification $\rightarrow$ real speedup $\approx 1.87\times$.
 - **Real gap from naive ceiling:** $1.87\times$ measured speedup, well short of the naive $k+1=5\times$ a surface reading might suggest.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Speedup} = \frac{E[\text{accepted}] \times 1}{k \times c_{\text{draft}} + 1}$$

where c_{draft} is the stated draft-pass cost relative to a verification pass — an explicit, separate assumption, not derived from α or k alone.

Production Perspective & Trade-offs

This two-step discipline matters directly for real production capacity planning — a team estimating speculative decoding's real benefit needs both a real, measured acceptance rate *and* a real, measured draft/target cost ratio on their actual hardware, not just one or the other.

Common Mistakes

1. Skipping Step 2 entirely and reporting $E[\text{accepted}]$ or the naive $k+1$ ceiling as if it were the real expected speedup.
2. Using an unrealistic or unstated draft-cost assumption — the real speedup figure is only as trustworthy as the real cost ratio it's built on.

COMMON FOLLOW-UP QUESTIONS

Q: How would a cheaper real draft model (lower c_{draft}) change this result?

A real, direct improvement — lowering c_{draft} shrinks the denominator, raising real expected speedup toward (but never reaching, at finite α) the naive $k+1$ ceiling.

Q: What real acceptance rate would be needed to double this example's speedup?

Would require solving the same two-step formula for a higher real α at the same $k = 4$, $c_{\text{draft}} = 0.2$ — a real, computable but nontrivial inversion, not a simple linear scaling of α .

One-Line Takeaway

Takeaway: At $\alpha=0.8$, $k=4$, a real, stated 0.2x draft-cost assumption gives a real expected speedup of $\approx 1.87x$ — genuinely positive but well short of the naive $5x$ ceiling, exactly because Step 2's real cost assumption matters.

Question 40: Why does speculative decoding specifically target the decode phase and not prefill?

[ESSENTIAL]

Conversational Answer

"Decode's real inefficiency — per Module 01's roofline framing — is that each expensive step reads the same real memory volume (weights, KV cache) regardless of whether it's producing an 'easy' or 'hard' token, since only one new token comes out of each pass. Speculative decoding directly exploits this: verifying several candidate tokens costs roughly the same real per-pass overhead as verifying one, so it amortizes that fixed real memory-traffic cost across more real output tokens per expensive pass. Prefill

already processes many real tokens in a single pass and is typically compute-bound rather than paying that same 'one token per expensive pass' tax — there's no equivalent real inefficiency there for speculative decoding's verification-amortization mechanism to address."

Intuitive Example

- Speculative decoding is like batching several small errands into one trip to make each trip more worthwhile — useful specifically because each individual trip (decode step) was expensive relative to how much it accomplished; a trip that already accomplishes a lot per visit (prefill) doesn't have that same problem to solve.

Key Interview Points

- **Decode's real inefficiency:** fixed per-step memory traffic regardless of how "easy" the token was — exactly what speculative decoding amortizes.
 - **Prefill's real situation:** already processes many tokens per pass, typically compute-bound — no equivalent inefficiency.
 - **Direct link to Module 01:** this is the same roofline distinction driving every other decode-specific optimization in this topic.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Directly follows from Question 3/4's arithmetic-intensity framing — decode's real memory-bandwidth-bound-ness at typical batch sizes is exactly the condition speculative decoding exploits.

Production Perspective & Trade-offs

This is a real, direct reason speculative decoding is deployed specifically for decode-heavy real workloads (long generations) rather than prompt-processing-heavy ones, where its real benefit would be structurally limited.

Common Mistakes

1. Assuming speculative decoding would help prefill too, by analogy — prefill lacks the specific "one token per expensive pass" inefficiency that gives speculative decoding its real leverage.
2. Overlooking that speculative decoding's real benefit is conditional on decode genuinely being memory-bandwidth-bound at the real deployment's batch size — per Question 4's own caveat, this isn't universal at every concurrency level.

COMMON FOLLOW-UP QUESTIONS

Q: Would speculative decoding's real benefit shrink at very high real batch sizes?

Plausibly — per Question 4, high concurrency can shift decode itself into a compute-bound regime, which would reduce the real memory-bandwidth-bound inefficiency speculative decoding specifically targets.

Q: Is there an equivalent "speculative prefill" technique?

Not in the same sense — prefill doesn't have the sequential-dependency structure that makes drafting-then-verifying meaningful the way it does for decode's genuinely-unknown-until-sampled next tokens.

One-Line Takeaway

Takeaway: Speculative decoding amortizes decode's real fixed per-step memory-traffic cost across multiple verified tokens — an inefficiency prefill, already compute-bound and multi-token-per-pass, doesn't share.

Question 41: Under what real conditions can speculative decoding produce a *negative* real speedup rather than a positive one?

[ESSENTIAL]

Conversational Answer

"When the real draft-model cost is too high relative to the real savings from amortizing verification passes. Question 39's own formula makes this concrete: if the real acceptance rate α is low, few draft tokens get accepted per round, so each round produces close to just the one bonus token — while still paying the real cost of k draft-model forward passes plus one verification pass. Module 07's own reference code demonstrated this directly: at the same $k=4$, $c_{\text{draft}}=0.2$ configuration but a much lower $\alpha=0.3$, real computed speedup dropped to 0.792x — a genuine, real slowdown, not just a smaller win. Speculative decoding is a real, conditional win, not an unconditional one."

Intuitive Example

- Sending an assistant ahead to draft several paragraphs before you review them only helps if the assistant usually gets things right — if they're wrong most of the time, you end up paying for their drafting effort *and* still having to write almost everything yourself anyway.

Key Interview Points

- **Real risk condition:** low real acceptance rate combined with non-negligible real draft-model cost.
- **Real verified example:** $\alpha=0.3$, same k/c_{draft} as the $\alpha=0.8$ case → real computed speedup of 0.792x, an actual slowdown.

- **General lesson:** speculative decoding's real benefit is conditional, not guaranteed.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same formula as Question 39 — evaluating it at a low real α can produce a computed speedup below 1.0, indicating a real, genuine slowdown rather than merely a smaller win.

Production Perspective & Trade-offs

This is a real, direct reason production speculative-decoding deployments require empirical validation of real acceptance rate on the *actual* target workload before committing — a draft model that performs well on one domain can genuinely underperform on another, with real negative consequences.

Common Mistakes

1. Assuming speculative decoding "can't hurt" since it preserves exact output correctness — correctness preservation says nothing about real speed; a real net slowdown is a genuine possible outcome, not a contradiction.
2. Deploying a draft model without validating its real acceptance rate against the specific real target workload/domain it will actually serve.

COMMON FOLLOW-UP QUESTIONS

Q: How would a real production team detect this failure mode before it causes a regression?

Real, direct empirical measurement of acceptance rate and end-to-end speedup on representative real workload samples before full deployment — exactly the two-step measurement discipline Question 39/42 describe.

Q: Does a higher real draft length k make this risk worse or better?

Worse at a low real α — more draft tokens proposed per round means more real wasted draft-model cost when acceptance is low, since rejected draft work still had to be computed.

One-Line Takeaway

Takeaway: A real, verified low- α case (0.3) produced a computed speedup of 0.792x — an actual slowdown, not just a smaller win — confirming speculative decoding is a real, conditional benefit, never an unconditional guarantee.

Question 42: A real notebook implemented and verified a correct speculative-decoding loop ($\alpha=0.4000$, matching the formula's predicted acceptance closely) but measured a genuine 0.365x end-to-end speedup — a real slowdown. Why don't these two real findings contradict each other, and why is the slowdown specifically attributable to that notebook's own unoptimized implementation and hardware, not a general property of speculative decoding?

[ESSENTIAL]

Conversational Answer

"Notebook 06 measured two genuinely separate things and reported both honestly. First, it implemented a manual speculative-decoding loop, independently verified correct — its output matched standard greedy decoding byte-for-byte — and measured a real empirical acceptance rate of $\alpha=0.4000$, closely matching Module 07's formula's own prediction of ≈ 1.65 expected accepted tokens per round at that α (the loop's real observed average was 40 accepted / 25 rounds = 1.6, very close). Second, and completely separately, it measured real end-to-end wall-clock speedup: 0.365x — a genuine slowdown. These don't contradict each other because acceptance rate and speedup are different real questions (Question 38's own point) — and this specific slowdown is directly attributable to *this implementation's* real, avoidable overhead: it called the draft model's `generate()` from scratch every round with no KV-cache reuse across rounds, and ran the target's verification with `use_cache=False`, recomputing the entire growing context from scratch every single round. A production-grade implementation that reuses KV caches on both sides would not pay that specific real cost — this result is honestly scoped to this exact reference implementation and hardware, not generalized as a property of speculative decoding itself."

Intuitive Example

- A chef who correctly follows a genuinely efficient recipe but insists on washing and re-sharpening every knife from scratch before each single step will still take longer overall than a chef using a less "correct-looking" workflow but reusing already-prepared tools — the recipe's real efficiency and the kitchen's real operational overhead are separate questions.

Key Interview Points

- **Real acceptance rate:** $\alpha=0.4000$, verified correct (byte-for-byte match with standard greedy), closely matching the formula's real prediction (1.6 observed vs. 1.650 predicted).
- **Real speedup:** 0.365x — a genuine, separately-measured slowdown.
- **Why no contradiction, and why scoped to this implementation:** the slowdown is directly attributable to this specific reference implementation's real, avoidable per-round overhead (no

draft KV-cache reuse, `use_cache=False` full recomputation) — not evidence about speculative decoding generally.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

This result is Question 41's mechanism observed directly: even a real, healthy acceptance rate cannot produce a positive real speedup if the real, actual per-round implementation cost (not the idealized c_{draft} assumption from Question 39) is high enough.

Production Perspective & Trade-offs

This is a real, concrete illustration of why a "correctness-first" reference implementation's timing should never be read as representative of a production-grade implementation's real performance — the two can differ dramatically due to real, addressable engineering overhead unrelated to the underlying technique's real merit.

Common Mistakes

1. Concluding from this real result that "speculative decoding doesn't work" — the result is honestly scoped to this specific unoptimized reference implementation and hardware, not a general claim about the technique.
2. Conflating a real, verified-correct acceptance rate with a real, positive speedup — Question 38 and this result both directly demonstrate these are separate real questions.

COMMON FOLLOW-UP QUESTIONS

Q: What specific real engineering change would most directly address this notebook's slowdown?

Reusing real KV caches across rounds for both the draft and target models (avoiding full-context recomputation each round) — the real, standard approach production speculative-decoding implementations use, deliberately omitted here for implementation transparency/correctness-verification simplicity.

Q: Why did the notebook implement this manually instead of using a library's built-in speculative decoding?

The library's built-in `assistant_model=`` API didn't expose a real, direct per-round acceptance count, so a manual, transparent loop was built specifically to measure real acceptance rate directly — at the real, acknowledged cost of forgoing the library's own real KV-cache-reuse optimizations.

One-Line Takeaway

Takeaway: A real, verified $\alpha=0.4000$ acceptance rate and a real 0.365x speedup don't contradict each other — the slowdown is honestly attributable to this specific reference implementation's

real, avoidable KV-cache-reuse overhead, not a general property of speculative decoding.

8. Inference Serving Engines & Production Architecture (Q43–Q48)

Question 43: Along which stable architectural dimensions should real serving engines (vLLM, TensorRT-LLM, TGI, llama.cpp) be compared, and why is a feature checklist a weaker basis for comparison?

[ESSENTIAL]

Conversational Answer

"A feature checklist changes release to release — engines converge on similar feature sets over time, so comparing them by 'does it have feature X' becomes stale quickly and doesn't reveal the real, underlying design philosophy. Stable architectural dimensions hold up better: real scheduling strategy (how requests get batched and admitted), real kernel-optimization approach (what compute techniques it relies on), real hardware support (which GPUs/platforms it targets), and real ease of multi-GPU scaling. These dimensions reflect genuine, durable design choices — vLLM and TGI prioritize real ease of deployment and broad model support; TensorRT-LLM prioritizes real peak NVIDIA-specific performance at the cost of portability; llama.cpp prioritizes real hardware breadth (CPU, Apple Silicon, consumer GPUs) over data-center-scale throughput."

Intuitive Example

- Comparing cars by which specific infotainment features they currently ship is far less durable than comparing them by their real underlying design philosophy — sports car, family minivan, off-road truck — since infotainment features converge across all of them over time while the fundamental design trade-offs persist.

Key Interview Points

- **Stable dimensions:** real scheduling strategy, kernel-optimization approach, hardware support, multi-GPU scaling ease.
 - **Why a feature checklist is weaker:** features converge over time; architectural philosophy is more durable.
 - **Real trade-off, not a ranking:** each engine's real design choice suits different deployment priorities, not a universal "best."
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a comparative/architectural survey, matching Module 08's own treatment.

Production Perspective & Trade-offs

Real engine selection should follow the actual real deployment constraints (available hardware, model-architecture breadth needed, portability requirements) rather than a fixed "best engine" assumption independent of context.

Common Mistakes

1. Ranking engines by a snapshot feature comparison that will be outdated within a few real release cycles.
2. Assuming one engine is universally "best" rather than recognizing each occupies a genuinely different, real point on the deployment-priority trade-off space.

COMMON FOLLOW-UP QUESTIONS

Q: Which engine would you pick for an edge/consumer-device deployment?

llama.cpp's real, broad hardware support (CPU, Apple Silicon, consumer GPUs) and quantization-first design specifically target that real deployment class, unlike the other three engines' data-center-oriented designs.

Q: Which architectural dimension matters most for a startup without dedicated NVIDIA infrastructure?

Real hardware support breadth and ease of deployment — TensorRT-LLM's NVIDIA-only real design would be a poor fit without dedicated NVIDIA infrastructure already in place.

One-Line Takeaway

Takeaway: Compare serving engines by durable architectural dimensions — scheduling, kernel approach, hardware support, multi-GPU scaling — not by a feature checklist that goes stale within a few real release cycles.

Question 44: Walk through the full real replica-level serving architecture — what does each layer (router, replica, per-replica scheduler, GPU) actually do?

[ESSENTIAL]

Conversational Answer

"Real requests first hit a request router/load balancer, which distributes them across multiple independent replicas — each replica being one real model-serving instance, possibly itself spanning multiple GPUs via tensor parallelism. Each replica has its own real per-replica scheduler, which decides real batch composition and continuous-batching admission for the requests routed to it. That scheduler's decisions get executed on that replica's own real GPU(s). The router's job is purely about *distributing load across replicas*; each replica's scheduler's job is about *managing what's happening within that one replica*, entirely independently of what any other replica is doing."

Intuitive Example

- A large restaurant chain's central reservation system (router) assigns diners to specific branch locations (replicas), and each branch's own floor manager (per-replica scheduler) independently decides seating and kitchen order within that branch, without needing to coordinate directly with other branches.

Key Interview Points

- **Router:** distributes real incoming requests across replicas.
 - **Replica:** one independent real model-serving instance (possibly itself multi-GPU via tensor parallelism).
 - **Per-replica scheduler:** manages real batch composition/admission within its own replica, independently of other replicas.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — architectural layering, verified conceptually via Question 47's real least-loaded routing worked example.

Production Perspective & Trade-offs

This layered structure is exactly why replica-level scaling (adding more replicas) and tensor/pipeline-parallel scaling (splitting one model across GPUs within a replica) are genuinely separate, composable real scaling axes, not competing choices.

Common Mistakes

1. Conflating the router's real job (distributing across replicas) with a per-replica scheduler's real job (managing within one replica) — they operate at genuinely different layers.
2. Assuming a single scheduler manages an entire multi-replica fleet — real per-replica schedulers operate independently, each managing only its own replica's real batch.

COMMON FOLLOW-UP QUESTIONS

Q: What real routing policy would the router use to pick a replica?

Real policies vary — least-loaded routing (Question 47's worked example) is one common, real approach, though round-robin and other policies exist too.

Q: Can one replica itself span multiple GPUs?

Yes — via tensor or pipeline parallelism (Question 45), when a single model doesn't fit on one GPU; that's a genuinely separate real concern from how many replicas exist.

One-Line Takeaway

Takeaway: The router distributes real requests across independent replicas; each replica's own scheduler manages batching within itself, entirely independently of every other replica — a real, layered architecture.

Question 45: Precisely distinguish replica-level scaling from tensor/pipeline-parallel scaling — what real problem does each solve?

[ESSENTIAL]

Conversational Answer

"Tensor and pipeline parallelism split *one* real model instance across multiple GPUs, specifically because that one model doesn't fit on a single GPU — a real *fitting* problem. Replica-level scaling runs *multiple independent copies* of a model (each possibly itself tensor-parallel) to serve more real concurrent traffic — a real *capacity* problem. These are genuinely different real scaling axes that compose together: a large model might need tensor parallelism just to fit on the available hardware at all, *and* multiple replicas of that same tensor-parallel group to handle real concurrent load beyond what one replica can serve."

Intuitive Example

- Needing a specialized team of surgeons (tensor parallelism) because one surgeon alone can't perform a particular complex operation is a genuinely different real problem from needing

multiple complete surgical teams (replicas) to handle more patients simultaneously.

Key Interview Points

- **Tensor/pipeline parallelism:** solves a real *fitting* problem — one model too large for one GPU.
 - **Replica-level scaling:** solves a real *capacity* problem — more concurrent traffic than one replica can serve.
 - **Composable:** real production deployments often need both simultaneously for large models under heavy real load.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a real, conceptual distinction directly building on Question 44's layered architecture.

Production Perspective & Trade-offs

Confusing these two real scaling axes leads to real, mis-provisioned deployments — adding more replicas doesn't help if the actual problem is that a single model instance doesn't fit on available hardware, and vice versa.

Common Mistakes

1. Treating "add more GPUs" as a single undifferentiated real lever without distinguishing whether the real goal is fitting a larger model or serving more concurrent traffic.
2. Assuming tensor parallelism alone can address a real capacity problem — it doesn't add serving capacity, it only enables one model instance to exist on hardware it otherwise couldn't fit on.

COMMON FOLLOW-UP QUESTIONS

Q: If a model already fits comfortably on one GPU, is tensor parallelism ever still useful?

Not for fitting purposes — but real pipeline-parallel-style splitting could still theoretically be used for other real engineering reasons, though typically replica-level scaling (Question 44) is the more direct real lever for handling additional concurrent capacity in that case.

Q: Does replica-level scaling require identical hardware across all replicas?

Not strictly, though real production deployments commonly do use matched hardware per replica for simplicity of scheduling and predictable real per-replica capacity.

One-Line Takeaway

Takeaway: Tensor/pipeline parallelism solves a real fitting problem (one model, too large for one GPU); replica-level scaling solves a real capacity problem (more traffic than one replica can serve) — genuinely different, composable axes.

Question 46: What is prefill/decode disaggregation, and what real trade-off does it introduce in exchange for reducing (not eliminating) interference between prefill and decode workloads sharing a pool?

[ESSENTIAL]

Conversational Answer

"Prefill and decode have genuinely different real compute/memory-bandwidth profiles — prefill is typically compute-bound, decode is typically memory-bandwidth-bound (with Question 4's own 'typical, not universal' caveat still applying). Running both phases on the same shared real GPU pool means a long real prefill can delay decode steps for other in-flight sequences sharing that pool — real head-of-line blocking. Prefill/decode disaggregation runs the two phases on *separate*, independently-scaled real GPU pools, transferring the computed KV cache from a prefill-pool GPU to a decode-pool GPU once prefill completes. This genuinely *reduces* — not eliminates — that interference, since each pool can now be sized and optimized for its own distinct bottleneck profile; the real trade-off it introduces in exchange is the added latency and system complexity of an explicit KV-cache transfer step between the two pools."

Intuitive Example

- Separating a restaurant's prep kitchen (long, batch-style cooking tasks) from its plating station (quick, sequential per-order tasks) into two distinct real work areas reduces one from blocking the other, but now requires a real, added hand-off step moving finished prep between the two areas.

Key Interview Points

- **Real problem addressed:** prefill's compute-bound work sharing a pool with decode's memory-bandwidth-bound work can cause real head-of-line blocking.
- **What disaggregation does:** separates the two onto independently-scaled real pools, genuinely *reducing* (not eliminating) that interference.
- **Real trade-off introduced:** an explicit, added real KV-cache-transfer latency and system-complexity cost.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — architectural pattern, connecting directly to Question 4's prefill/decode roofline distinction.

Production Perspective & Trade-offs

Disaggregation's real transfer cost needs to be small relative to the head-of-line-blocking risk it reduces to be a net real win — genuinely not automatically worthwhile at every deployment scale or traffic pattern (Question 48).

Common Mistakes

1. Claiming disaggregation "eliminates" head-of-line blocking entirely — the real, accurate claim is that it reduces interference between the two specific workload types, not a complete elimination of every possible contention source.
2. Ignoring the real, added KV-cache-transfer cost when evaluating whether disaggregation is worthwhile for a given real deployment.

COMMON FOLLOW-UP QUESTIONS

Q: What real signal would suggest disaggregation is worth its added complexity?

Real, observed head-of-line blocking between prefill and decode workloads sharing a pool — measurable as decode-latency degradation correlated with concurrent long real prefill requests.

Q: Does disaggregation require any change to Module 08's replica-level architecture?

It extends it — instead of one pool type per replica, disaggregation introduces two real, distinct pool types (prefill and decode) with an explicit real transfer mechanism between them, layered on top of the same router/replica/scheduler structure.

One-Line Takeaway

Takeaway: Prefill/decode disaggregation reduces — not eliminates — real interference between the two workloads' differing bottleneck profiles, at the real cost of an added KV-cache-transfer step and system complexity.

Question 47: A least-loaded routing simulation, started from an uneven queue-depth spread and run for 6 new requests, narrowed but did not eliminate that spread. Since this was a simulation and not a production

routing experiment, what does the result still reveal about the limits of correct load-balancing logic alone?

[ESSENTIAL]

Conversational Answer

"Module 08's simulation started with a deliberately uneven real starting queue-depth spread — $\{5, 2, 8\}$ across three replicas — and routed 6 new simulated requests, each correctly assigned to whichever replica was least-loaded at that exact moment. The spread narrowed from 6 down to 2, but didn't reach zero. Since this was a simulation, not an actual multi-replica production deployment, it doesn't claim to have measured real production routing behavior — but the routing *logic* itself was genuinely correct at every single step, and the result still reveals something real about that logic's limits: a short burst of requests isn't always enough to fully rebalance a real starting imbalance, especially when the most-loaded replica never happens to dip below the others during that specific window. Correct least-loaded logic narrows imbalance; it doesn't guarantee it eliminates imbalance within any given short window."

Intuitive Example

- Correctly directing each new customer to whichever checkout line is currently shortest will narrow an initially very uneven set of line lengths over time, but a short burst of new customers isn't guaranteed to fully equalize lines that started out dramatically uneven.

Key Interview Points

- **Simulation, not production:** correctly labeled as such — no real multi-replica deployment was involved.
 - **Logic was correct at every step:** least-loaded routing genuinely picked the currently-lowest-load replica each time.
 - **What the simulated result still reveals:** correct logic narrows (6→2) but doesn't guarantee eliminating imbalance within a short window.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — a real, direct simulation trace, not a derived quantity.

Production Perspective & Trade-offs

This is a genuine, useful caution for real production monitoring: observing residual load imbalance after a load balancer has been running doesn't necessarily indicate the routing logic itself is broken —

it may simply reflect a real, still-recovering starting imbalance or an ongoing burst pattern.

Common Mistakes

1. Describing this result as "a real production routing experiment" — it was explicitly a simulation; conflating the two overstates what was actually demonstrated.
2. Concluding from residual imbalance that the least-loaded logic is flawed — the logic was verified correct at every individual routing decision; residual imbalance is a real, separate consequence of a short observation window and a skewed starting point.

COMMON FOLLOW-UP QUESTIONS

Q: What would need to change to more fully close this simulated gap?

More simulated routing decisions over a longer window, or a starting point with less initial skew — the *logic* itself doesn't need to change, since it was already behaving correctly at every step.

Q: Does this finding generalize to a real production system exactly as measured here?

Not directly — this was a simulation with a specific, illustrative starting imbalance and request count; a real production system's actual behavior would depend on its own real traffic patterns, though the qualitative lesson (correct logic doesn't guarantee instant full rebalancing) plausibly transfers.

One-Line Takeaway

Takeaway: A simulation — not a production experiment — showed correct least-loaded routing narrowing (not eliminating) a starting imbalance from 6 to 2 within a short window, revealing a real limit of correct logic alone, not a flaw in the logic itself.

Question 48: When might tensor parallelism and replica-level scaling both be needed simultaneously for the same real deployment?

[ESSENTIAL]

Conversational Answer

"Exactly when a real deployment faces both problems Question 45 distinguished at once: a model large enough that it genuinely doesn't fit on a single GPU — requiring tensor parallelism just to exist on the available hardware — *and* real concurrent traffic beyond what one instance of that (already multi-GPU) model can serve — requiring multiple replicas of that same tensor-parallel group. A large real production model serving high real traffic is the canonical case: each replica might itself span, say, 4 GPUs via tensor parallelism just to fit, and the deployment then runs several such replicas to meet

real demand — two genuinely separate scaling problems, both actually present, both needing to be solved."

Intuitive Example

- A surgical procedure that both requires a full specialized team (not one surgeon alone can do it) *and* needs to be performed on many real patients simultaneously requires both assembling complete teams (tensor parallelism) and running multiple such teams in parallel (replicas) — solving one problem doesn't solve the other.

Key Interview Points

- **When both are needed:** a real model too large for one GPU (fitting problem) *and* real traffic beyond one instance's capacity (capacity problem), simultaneously.
 - **Real production pattern:** each replica itself spans multiple GPUs (tensor-parallel), and the deployment runs several such replicas.
 - **Genuinely separate, composable:** solving the fitting problem alone doesn't address the capacity problem, and vice versa.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — a direct, real composition of Question 45's two distinct scaling axes.

Production Perspective & Trade-offs

Real large-model production deployments (frontier-scale models serving real, heavy traffic) routinely need exactly this combination — the two scaling axes aren't a theoretical curiosity, they're the standard real pattern at that deployment scale.

Common Mistakes

1. Assuming tensor parallelism alone (without additional replicas) can handle arbitrarily high real concurrent traffic — it solves fitting, not capacity.
2. Assuming replica-level scaling alone (without tensor parallelism) can serve a model too large for one GPU — replicating an instance that doesn't fit doesn't solve the fitting problem.

COMMON FOLLOW-UP QUESTIONS

Q: How would you decide the real tensor-parallel degree (how many GPUs per replica) for a given real model?

The minimum real number of GPUs needed for the model to genuinely fit (weights plus a real working KV-cache budget) — over-splitting adds real communication overhead between GPUs within a replica without additional benefit.

Q: Does adding more replicas of an already tensor-parallel model change each replica's own internal behavior?

No — each replica's internal tensor-parallel structure operates independently and identically; replica count is purely a capacity lever layered on top, per Question 44's architecture.

One-Line Takeaway

Takeaway: A real deployment needs both tensor parallelism and replica-level scaling when it faces a real fitting problem (model too large for one GPU) and a real capacity problem (traffic beyond one instance) simultaneously — the standard pattern at large-model production scale.

9. Production Monitoring, Cost Modeling & Latency Optimization (Q49–Q54)

Question 49: Why is summing each pipeline stage's own p99 latency not a mathematically valid way to derive true end-to-end p99 latency?

[ESSENTIAL]

Conversational Answer

"p99 of a sum of real random variables isn't generally the sum of each variable's own p99 — that's a real, mathematical fact, not a rule of thumb. Two real reasons: the components typically aren't independent — a real, busy system tends to have correlated queue-wait and decode delays, for instance, since whatever's driving one up (system load) often drives the other up too — and tail probabilities genuinely don't add linearly even when independence does hold. So treating Module 09's additive latency-budget formula as an *exact* derivation of end-to-end p99 would be a real, mathematical overclaim. It's deliberately framed instead as a practical budgeting *tool* — useful for reasoning about which real component to optimize first, not for deriving an exact tail-latency figure."

Intuitive Example

- Adding up the "worst 1% delay" for each leg of a multi-leg flight itinerary doesn't give you the real worst-1%-of-the-time total trip delay — bad weather that delays one leg often correlates with

delays on connecting legs too, and the real combined tail risk doesn't add up as simply as summing each leg's own tail figure.

Key Interview Points

- **Real mathematical fact:** p99 of a sum $\neq$ sum of each term's own p99, in general.
 - **Two real reasons:** components aren't independent in a real busy system; tail probabilities don't add linearly even under independence.
 - **Explicit framing:** Module 09's formula is a real budgeting tool, not an exact p99 derivation.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula in this question specifically — the budgeting formula itself is Question 50/51's focus.

Production Perspective & Trade-offs

Real production monitoring should track true end-to-end p99 latency directly from real, measured request traces — not attempt to reconstruct it by summing individually-measured per-stage p99 figures, which would be a real, mathematically unsound approach.

Common Mistakes

1. Presenting a summed per-stage p99 figure as "the real end-to-end p99 latency" in an interview or a real production dashboard — a genuine, common, mathematically incorrect practice this question directly corrects.
2. Assuming independence between pipeline stages is a safe simplifying assumption for a real, busy production system — correlated load is the real, typical case, not the exception.

COMMON FOLLOW-UP QUESTIONS

Q: How should real end-to-end p99 latency actually be measured in production?

Directly, from real, complete request traces spanning the full pipeline — measuring the real, actual end-to-end distribution rather than reconstructing it from independently-measured per-stage percentiles.

Q: Is there any real, valid use for per-stage p99 figures at all?

Yes — for identifying which real stage is disproportionately contributing to tail latency (a diagnostic use), just not for mathematically deriving the exact combined figure.

One-Line Takeaway

Takeaway: p99 of a sum isn't the sum of each term's own p99 — real correlated load and nonlinear tail-probability addition both break that assumption, which is exactly why Module 09's

formula is framed as a budgeting tool, not an exact derivation.

Question 50: Walk through the approximate latency-budget decomposition — what is it actually useful for, given it isn't an exact derivation?

[ESSENTIAL]

Conversational Answer

"Even though it's explicitly not a mathematically exact p99 derivation (Question 49), the additive latency-budget decomposition — queue wait plus prefill plus decode plus network/serialization — is genuinely useful as a *practical prioritization tool*. It tells you, at a glance, which real component of the request lifecycle is contributing the most to total latency, so engineering effort goes toward whichever term actually dominates rather than whichever is easiest to optimize or best understood. Module 09's own real worked example made this concrete: decode accounted for 76.2% of the illustrative budget, and a real, notebook-filled-in version found decode's real share even higher, at 97.48% — in both cases, the decomposition's real value was pointing directly at where to focus, not producing a mathematically exact tail-latency number."

Intuitive Example

- A rough budget breakdown of where a household's money goes each month — even if it doesn't capture every real correlation between spending categories precisely, it's genuinely useful for deciding which category to focus on cutting first.

Key Interview Points

- **Explicit scope:** a practical budgeting/prioritization tool, not an exact p99 derivation (Question 49).
- **Real value:** identifies which component dominates, directing real optimization effort.
- **Real example:** decode dominated at both 76.2% (illustrative) and 97.48% (real-data-filled) versions.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Latency}_{\text{budget}} \approx \text{Queue}_{\text{wait}} + \text{Prefill}_{\text{time}} + \text{Decode}_{\text{time}} + \text{Network/Serialization}$$

Production Perspective & Trade-offs

Real production teams use exactly this kind of decomposition to decide where to invest optimization effort first — GPU-level optimization alone, for instance, doesn't address a real queue-wait-dominated

budget, a genuine, real distinction this decomposition surfaces directly.

Common Mistakes

1. Treating this decomposition as the final answer to "what is our real p99 latency" rather than as a tool for deciding what to optimize.
2. Ignoring the queue-wait and network terms entirely because they're often "assumed" values rather than measured — they still matter for the real prioritization exercise, even when not directly measurable in a given environment.

COMMON FOLLOW-UP QUESTIONS

Q: What would this decomposition suggest optimizing first if network/serialization dominated instead of decode?

Network/serialization-focused engineering effort (payload size, connection handling) rather than GPU-level decode optimization — the whole point of the decomposition is that the dominant term should guide where real effort goes.

Q: Does this decomposition apply identically across every real deployment?

The additive structure is general, but the real relative magnitude of each term is genuinely workload- and deployment-specific — Module 09's own real notebook-filled version found a different (more extreme) decode share than the illustrative example, underscoring this point directly.

One-Line Takeaway

Takeaway: The latency-budget decomposition's real value is prioritization — identifying which component dominates so optimization effort targets the actual bottleneck, not producing a mathematically exact p99 figure.

Question 51: Given real or assumed values for queue wait, prefill, decode, and network/serialization, compute a full latency budget and identify the dominant term.

[ESSENTIAL]

Conversational Answer

"Module 09's own real, filled-in capstone example: queue wait held at the module's stated assumption of 150ms (not measurable in a single-machine notebook context), real measured prefill (TTFT) of 156.49ms, real measured decode of 13,805.80ms for 100 real output tokens (using the real TPOT floor of 138.058ms/token), and network held at the stated assumption of 50ms. Total budget comes to

14,162.29ms, with decode accounting for 97.48% of that total — even more extreme than the module's original illustrative example, which had decode at 76.2%. In both cases, decode is overwhelmingly the dominant term, directly pointing at decode-phase optimization (quantization, speculative decoding, batching) as the highest-leverage lever, not queue wait or network."

Intuitive Example

- Filling in a household budget with a mix of real receipts (decode, prefill) and reasonable estimated figures (queue wait, network, since you don't have exact records for those) still gives you a genuinely useful picture of which category dominates your real spending.

Key Interview Points

- **Real-data-filled example:** 150ms (assumed) + 156.49ms (real) + 13,805.80ms (real) + 50ms (assumed) = 14,162.29ms total.
 - **Dominant term:** decode, at 97.48% of the total.
 - **Real, direct implication:** decode-phase optimization is the highest-leverage lever in this specific example.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Same additive formula as Question 50, evaluated with a mix of real measured and stated-assumption inputs — explicitly labeled which terms are which.

Production Perspective & Trade-offs

Mixing real measured values with honestly-labeled assumptions (for genuinely unmeasurable terms in a given environment) is a real, legitimate practice — as long as the distinction between "measured" and "assumed" stays explicit, which this exact example preserved throughout.

Common Mistakes

1. Presenting an assumption-filled term (queue wait, network) as if it were a real measured value — the honest labeling distinction matters for interpreting the result correctly.
2. Computing the total without checking whether the dominant term actually matches real intuition about the workload — a sanity check worth doing before trusting the breakdown.

COMMON FOLLOW-UP QUESTIONS

Q: Why couldn't queue wait and network be measured directly in this real example?

A single-notebook, single-machine environment has no real network stack or request queue to measure against — those terms remained the module's original stated assumptions rather than being fabricated as measured values.

Q: How would you extend this real example to a genuinely multi-machine production deployment?

Replace the assumed queue-wait and network terms with real, measured values from an actual deployed request-queue and network stack, while keeping the real prefill/decode measurement approach unchanged.

One-Line Takeaway

Takeaway: A real, mostly-measured worked example found decode dominating at 97.48% of a 14,162.29ms total budget — an even more extreme dominance than the module's illustrative 76.2%, reinforcing decode as the highest-leverage optimization target.

Question 52: Derive the GPU-time-based cost model and compute cost-per-request and cost-per-token from a real GPU-time and cost-rate example — why does this differ from an API token-price model?

[ESSENTIAL]

Conversational Answer

"Unlike an API token-price model — a hosted provider's price sheet, reflecting whatever margin and packaging decisions that provider made — the GPU-time-based cost model targets the actual underlying *infrastructure* cost: how much real GPU time a request consumes, multiplied by a real GPU cost rate. Module 09's own real, filled-in example: real GPU-active time (prefill plus decode, explicitly excluding queue wait and network since the GPU isn't necessarily occupied by this specific request during those) came to 13.96229 seconds for a 100-output-token request. At an illustrative *2.00/hourrate*, that's a real cost — per — request of 0.007757, and a real cost-per-token of \$0.00007757. This model is the right one for understanding *your own infrastructure's* real cost structure — an API price model instead reflects a hosted provider's pricing decisions, which may or may not track the real underlying compute cost directly."

Intuitive Example

- Computing the real cost of driving your own car based on actual fuel and maintenance costs is a genuinely different calculation from looking up a taxi service's posted per-mile fare — the taxi fare bundles in the driver's margin and business decisions, not just the real underlying cost.

Key Interview Points

- **Formula:** $\text{Cost}_{\text{request}} = \text{GPU_time}_{\text{request}} \times \text{GPU_cost_rate}$, with cost-per-token derived from that.
 - **Real example:** 13.96229s real GPU time $\rightarrow 0.007757/\text{request} \rightarrow 0.00007757/\text{token}$ at an illustrative \$2.00/hr rate.
 - **Why it differs from API pricing:** targets real underlying infrastructure cost, not a hosted provider's price-sheet decisions.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

$$\text{Cost}_{\text{request}} = \text{GPU_time}_{\text{request}} \times \text{GPU_cost_rate} \qquad \text{Cost}_{\text{per_token}} = \frac{\text{Cost}_{\text{request}}}{N_{\text{tokens_generated}}}$$

Production Perspective & Trade-offs

This is the real, appropriate cost model for a team running its own real infrastructure and needing to understand its actual per-request/per-token cost — as distinct from a team consuming a hosted API, where the provider's price sheet is the relevant real number instead.

Common Mistakes

1. Using a hosted API's per-token price to estimate self-hosted infrastructure cost, or vice versa — these are genuinely different real cost models answering different real questions.
2. Including queue-wait or network time in the real GPU-time calculation — those don't represent time the GPU was actually occupied by the specific request in question.

COMMON FOLLOW-UP QUESTIONS

Q: Why exclude queue wait and network time from the real GPU-time figure specifically?

Because the GPU isn't necessarily occupied by *this specific request* during queue wait or network transfer — including them would overstate the real GPU-time cost actually attributable to serving that request's compute.

Q: How would this real cost model change under a disaggregated prefill/decode architecture (Question 46)?

Real GPU time would need to be tracked and potentially cost-rated separately across the two distinct real pools, since each pool may have different real hardware/utilization characteristics.

One-Line Takeaway

Takeaway: A real GPU-time-based cost model ($0.007757/\text{request}$, $0.00007757/\text{token}$ in this example) reflects actual infrastructure cost — genuinely different from an API token-price model, which reflects a hosted provider's own pricing decisions.

Question 53: Name this topic's full observability metric set — TTFT, TPOT, throughput, GPU compute utilization, memory/HBM utilization, KV-cache utilization, queue depth, and p95/p99 latency — and explain what distinct real failure mode each one catches that the others would miss.

[ESSENTIAL]

Conversational Answer

"Each of these eight real metrics catches a genuinely different real failure mode a production inference system can hit. TTFT and TPOT catch real prefill-phase and decode-phase latency regressions specifically — distinguishing which phase degraded. Throughput catches an overall real capacity/efficiency regression. GPU compute utilization catches real compute underutilization (wasted hardware capacity). Memory/HBM utilization catches real memory-bandwidth-bound bottlenecks. KV-cache utilization specifically catches the real risk Module 02 established — cache growth outpacing available memory, a genuine precursor to OOM. Queue depth catches real request-admission/scheduling backlog before it manifests as user-visible latency. p95/p99 latency catches real tail-latency degradation that an average-latency metric alone would completely hide. No single metric substitutes for another — that's exactly why the set is deliberately broad."

Intuitive Example

- A car's dashboard has separate gauges for speed, fuel, engine temperature, and oil pressure precisely because each one catches a genuinely different real failure mode — a single combined "car health" gauge would hide which specific system was actually in trouble.

Key Interview Points

- **The full set:** TTFT, TPOT, throughput, GPU compute utilization, memory/HBM utilization, KV-cache utilization, queue depth, p95/p99 latency.
- **Deliberately broad:** each metric catches a real failure mode the others would miss.
- **KV-cache utilization specifically:** the real, direct operational link to Module 02's OOM-risk concern.

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

No formula — this is a monitoring-coverage argument, directly connecting each metric back to the specific module/mechanism it operationalizes.

Production Perspective & Trade-offs

Real production incident response benefits directly from this breadth — a real regression's root cause (queue backlog vs. KV-cache growth vs. GPU underutilization) is diagnosable much faster with all eight tracked separately than with any single blended "health" metric.

Common Mistakes

1. Monitoring only aggregate throughput or average latency, missing real tail-latency degradation (p95/p99) or a real, slow-building KV-cache-driven OOM risk that an average wouldn't reveal until it's already critical.
2. Treating GPU compute utilization alone as sufficient — a real, healthy compute-utilization number can coexist with a real bottleneck elsewhere (queue wait, memory bandwidth) that compute utilization alone wouldn't surface.

COMMON FOLLOW-UP QUESTIONS

Q: If GPU compute utilization looks healthy but real p99 latency is still breaching SLO, what would you check next?

Real queue depth and queue-wait time specifically — a healthy compute-utilization number rules out compute as the bottleneck, pointing instead toward queueing or (under a disaggregated architecture) an imbalance between prefill-pool and decode-pool load.

Q: Which metric most directly warns of an impending real OOM from KV-cache growth?

KV-cache utilization, tracked against Module 02's real memory-footprint formula — a metric specifically designed to catch this exact real failure mode before it manifests as an actual crash.

One-Line Takeaway

Takeaway: Eight distinct real metrics — TTFT, TPOT, throughput, GPU/memory/KV-cache utilization, queue depth, p95/p99 latency — each catch a genuinely different real failure mode; no single one substitutes for the others.

Question 54: (synthesis) A real notebook filled in the latency-budget decomposition with genuinely measured prefill/decode numbers instead of

illustrative ones, finding decode's real share even higher (97.48%) than the illustrative example (76.2%). Design an end-to-end production inference stack for a new LLM feature — KV-cache/batching strategy, quantization target, serving architecture, and monitoring plan — and explain when and why decode-phase optimization should be prioritized first: specifically when real profiling shows it dominates the latency budget, as this topic's own repeated real measurements found, rather than as a universal default.

[ESSENTIAL]

Conversational Answer

"Starting from real profiling, not assumption: given this topic's own repeated real finding that decode dominates the latency budget in these specific tested workloads (76.2% illustrative, 97.48% real-measured), the design should begin by *confirming* that same pattern holds for the new feature's actual real workload — via Module 09's decomposition applied to real, measured numbers, not assumed to transfer automatically. If real profiling confirms decode dominance, prioritize decode-phase levers: GQA/MQA in the model architecture if choosable (Module 02), KV-cache-aware batching with continuous batching plus PagedAttention (Modules 03/06), and quantization validated empirically on the target hardware rather than assumed to help — since Notebook 04's own real finding showed quantization can genuinely backfire on decode latency specifically. Serving architecture should use replica-level scaling for capacity plus tensor parallelism only if the model genuinely doesn't fit on one GPU, with disaggregation considered only if real head-of-line blocking is actually observed. Monitoring should track the full eight-metric set (Question 53), with KV-cache utilization and TPOT specifically watched given the decode-dominant hypothesis. Deliberate MVP scope cuts: skip disaggregation and aggressive quantization initially, validate them empirically once real production traffic data confirms they're warranted — building on evidence, not assumption, exactly as this topic's own notebooks did throughout."

Intuitive Example

- A doctor treating a new patient starts by actually examining *this* patient's real symptoms before prescribing treatment, even if most similar patients they've seen before had a particular condition — the pattern from prior cases informs where to look first, but doesn't replace confirming it for this specific real patient.

Key Interview Points

- **Start with real profiling:** confirm decode-dominance for *this* workload via Module 09's decomposition, don't assume it transfers from prior examples.
- **If confirmed, prioritize decode-phase levers:** GQA/MQA, continuous batching + PagedAttention, empirically-validated quantization (not assumed).

- **MVP scope discipline:** defer disaggregation/aggressive quantization until real production data actually warrants them.
-

[DEEP DIVE]

Technical Intuition & Key Formulas (No Derivations)

Synthesizes every prior formula in this topic — none re-derived here; the point of this question is applying them in the correct real order (profile first, then optimize the confirmed bottleneck).

Production Perspective & Trade-offs

This question's entire point is that "optimize decode first" is a real, evidence-based conclusion from this topic's own repeated measurements on *these specific* tested workloads — not a universal rule to apply blindly to every new deployment without first checking whether the same pattern actually holds there.

Common Mistakes

1. Applying "decode dominates, so optimize decode first" as a universal starting assumption without first running the real Module 09 profiling exercise on the new feature's own actual workload.
2. Designing a maximally-optimized stack (disaggregation, aggressive quantization, every technique from every module) for an MVP, rather than deliberately deferring techniques until real production evidence warrants them.

COMMON FOLLOW-UP QUESTIONS

Q: What would change in this design if real profiling instead showed prefill dominating for this specific new feature?

The priority would shift toward prefill-phase levers instead — real compute throughput and prompt-length management — since the entire premise of decode-first prioritization is conditional on real profiling confirming decode's dominance for the workload in question, not an unconditional default.

Q: Name two real cases from this topic's own notebooks where a technique that "sounds" like an improvement measurably didn't help, or actively hurt.

Quantization's real memory savings producing a net latency **loss** on this specific small-model/consumer-GPU setup (Notebook 04), and the manual speculative-decoding reference implementation's real, verified-correct acceptance rate coexisting with a genuine net **slowdown** due to its own unoptimized KV-cache handling (Notebook 06) — both real, honestly-reported findings that a naive "more sophisticated technique = faster" assumption would have missed.

One-Line Takeaway

Takeaway: Decode-phase optimization deserves priority specifically when real profiling confirms it dominates the latency budget — as this topic's own repeated real measurements found on these tested workloads — not as a universal default applied without first checking.

Final Revision Sheet

Quick-Recall Table

#	Question	One-Line Answer
1	Why is decoding sequential?	Token $t+1$ genuinely requires token t to exist; inference batches across requests, not within one.
2	What is the roofline model?	Arithmetic intensity (FLOPs/byte) vs. the hardware's ridge point — tells you which bottleneck (compute or bandwidth) actually governs.
3	Compute ridge point, decode/prefill intensity	Ridge ≈ 153 ; decode $I \approx 1.0$ (memory-bound); prefill(512 tok) $I \approx 512$ (compute-bound).
4	Is "decode memory-bound" universal?	Typical, not universal — 256 concurrent decode steps real-verified to push I to 256, above the ridge.
5	TTFT vs. TPOT	TTFT scales with prompt length (prefill); TPOT is a largely fixed per-step decode cost.
6	Real $\approx 897\times$ TTFT/TPOT gap	Raw TTFT looked flat; converting to cost-per-prompt-token revealed prefill is $\approx 897\times$ cheaper per token than decode.
7	What does KV cache store?	Per-layer K/V for prior tokens; caching means only the new token's K/V is computed each step.
8	Why N_{KV_heads} , not query heads?	Only K/V get cached; GQA/MQA share fewer real KV heads across more query heads.
9	Real MHA vs. GQA memory example	64.0GB (MHA) $\rightarrow$ 16.0GB (GQA, 8 vs. 32 KV heads) — exact real 4x reduction.

#	Question	One-Line Answer
10	When does KV cache dominate memory?	Workload-dependent — real low concurrency/short context: weights dominate; high concurrency/long context: KV cache dominates.
11	Real $\approx 26\times$ memory_allocated() gap	Root-caused to the real logits tensor (93.9%-96.1% of the gap), not the KV cache itself.
12	Why does GQA/MQA preserve quality?	Only shared K/V projections reduced; query-head diversity (attention scoring) untouched.
13	Does PagedAttention compress KV cache?	No — improves real allocation/utilization/sharing; per-token cost is unchanged.
14	Why does contiguous allocation waste memory?	Reserves worst-case length per sequence since real final length is unknown in advance.
15	Real contiguous vs. paged waste example	80.5% (contiguous) $\rightarrow$ 1.8% (paged) — a real, code-verified $\approx 44\times$ reduction.
16	Paged waste bound	Bounded by (block_size - 1) per sequence under a simple block model; contiguous waste is not similarly bounded.
17	How does paging enable sharing?	Blocks are independently addressable — multiple sequences can reference the same real shared-prefix blocks.
18	Real-grounded paged simulation	Real straggler lengths $\rightarrow$ 68.75% $\rightarrow$ 10.71% ($\approx 6.4\times$) simulated waste reduction — stayed labeled a simulation.
19	Why "IO-aware," not FLOP-reduction?	Same real FLOPs as naive attention; the real win is reduced HBM traffic.
20	Why does tiling reduce HBM traffic?	Avoids materializing the full score matrix in HBM; tiles stay in fast SRAM with online softmax.
21	Real HBM-access-ratio hand calc	$\approx 3.0\times$ (N=8) $\rightarrow$ $\approx 17.0\times$ (N=64) — the reduction ratio grows with sequence length.
22	FlashAttention vs. PagedAttention	FlashAttention: compute-kernel IO within one pass. PagedAttention: memory management across requests. Complementary.

#	Question	One-Line Answer
23	Same FLOPs, different latency — why?	Real wall-clock time is often memory-bandwidth-bound, not FLOP-bound.
24	Real Notebook 03 finding	FlashAttention genuinely unavailable on this build; EFFICIENT_ATTENTION vs. MATH real latency gap grew $\approx 5.34x \rightarrow \approx 33.07x$.
25	Three quantization targets	Weights, activations, KV cache — each with its own real accuracy/speed/memory trade-off.
26	Bytes-per-parameter formula	$\text{Mem} = N_{\text{params}} \times \text{bits}/8$ — identical across all three targets.
27	Real FP16/INT8/INT4 memory example	Weights 13.04/6.52/3.26GB; KV cache 64.00/32.00/16.00GB; combined 77.04 $\rightarrow$ 19.26GB (real 4x).
28	Why doesn't memory win guarantee speed win?	Needs (1) genuinely memory-bandwidth-bound workload and (2) genuine native low-precision kernel support.
29	Why is KV-cache quantization riskier?	No offline calibration (unlike weights); errors can compound across a long real generation.
30	Real Notebook 04 finding	Genuine 36.2%/54.3% memory savings but 3.37x/1.34x real latency <i>slowdowns</i> — memory win, net loss.
31	Why does static batching waste compute?	No slot frees until the whole batch's slowest member finishes.
32	Continuous batching mechanism	Freed slots backfilled immediately with new real waiting requests — no fixed group.
33	Real static vs. continuous waste example	54.4% $\rightarrow$ 13.75% (real 4.0x reduction) — with an honest, nonzero leftover-queue gap.
34	Is "always improves both" true?	Overclaim — throughput generally improves; per-request latency is genuinely scheduler/workload-dependent.
35	Why pair continuous batching with PagedAttention?	Constant slot turnover needs cheap, frequent (re)allocation — exactly what paging provides.
36	Real Notebook 05 straggler event	6.36x throughput scaling (bs1 $\rightarrow$ 4), then a real 278% latency jump at bs=8 from 2 real non-stopping sequences.

#	Question	One-Line Answer
37	Why is verification parallel?	All k candidate positions' inputs are already known — one target forward pass checks them all.
38	Expected-accepted-tokens formula	$E[\text{accepted}] = (1 - \alpha^{k+1}) / (1 - \alpha)$ — expected tokens, NOT the complete speedup.
39	Real two-step speedup example	$\alpha=0.8, k=4 \rightarrow E[\text{accepted}] \approx 3.362$; with a stated 0.2x draft-cost assumption $\rightarrow$ real speedup $\approx 1.87x$.
40	Why decode, not prefill?	Decode pays a fixed per-step memory-traffic tax regardless of token count produced; prefill doesn't.
41	When can speedup go negative?	Low real α combined with non-negligible draft cost — real verified example: $\alpha=0.3 \rightarrow 0.792x$ (a slowdown).
42	Real Notebook 06: $\alpha=0.4$ but 0.365x speedup	Acceptance and speedup are separate real questions; slowdown scoped to this implementation's own unoptimized KV-cache handling.
43	How to compare serving engines	Stable dimensions (scheduling, kernel approach, hardware support, multi-GPU scaling) — not a feature checklist.
44	Replica-level architecture layers	Router $\rightarrow$ replicas $\rightarrow$ per-replica scheduler $\rightarrow$ GPU(s), each layer with a distinct real job.
45	Replica scaling vs. tensor parallelism	Tensor/pipeline parallel: real fitting problem. Replica scaling: real capacity problem.
46	What does disaggregation trade off?	Reduces (not eliminates) prefill/decode interference, at the real cost of an added KV-cache-transfer step.
47	Real least-loaded routing simulation	A simulation (not production) — narrowed a real starting spread from 6 to 2, didn't eliminate it.
48	When are both scaling axes needed?	When a real model doesn't fit on one GPU AND real traffic exceeds one instance's capacity, simultaneously.
49	Why can't you sum per-stage p99?	Real components aren't independent, and tail probabilities don't add linearly.
50	What is the latency budget useful for?	A real prioritization tool — identifies the dominant term, not an exact p99 derivation.

#	Question	One-Line Answer
51	Real filled-in latency budget example	Decode = 97.48% of a real 14,162.29ms total — even more extreme than the illustrative 76.2%.
52	GPU-time cost model vs. API pricing	Real infrastructure cost (GPU-time × rate) vs. a hosted provider's own price-sheet decisions.
53	The 8-metric observability set	TTFT, TPOT, throughput, GPU/memory/KV-cache utilization, queue depth, p95/p99 latency — each catches a distinct real failure mode.
54	(<i>synthesis</i>) When to optimize decode first	When real profiling confirms it dominates the latency budget — an evidence-based conclusion, not a universal default.

Essential Formula Cheat Sheet

Formula	Meaning
$I = \text{FLOPs/Bytes moved}; I_{\text{ridge}} = \text{Peak FLOPs/s/Peak Bandwidth}$	Roofline arithmetic intensity and ridge point (Q2-4)
$\text{Mem}_{\text{KV}} = 2BLN_{\text{layers}}N_{\text{KV_heads}}d_{\text{head}}\text{bytes}_{\text{dtype}}$	KV-cache memory footprint (Q8-11)
Contiguous waste = $\text{max_len} \times n - \sum L_i$; Paged waste = $\sum (\lceil L_i/\text{block} \rceil \times \text{block} - L_i)$	Allocation waste (Q15-16, 18)
$\text{HBM}_{\text{naive}} \approx 4N^2 + 4Nd; \text{HBM}_{\text{tiled}} \approx 4Nd$	Simplified IO-access comparison (Q21) — intuition-level, not exact
$\text{Mem} = N_{\text{params}} \times \text{bits}/8$	Bytes-per-parameter quantization (Q26-27, 30)
$E[\text{accepted}] = (1 - \alpha^{k+1})/(1 - \alpha)$	Expected accepted tokens under simplified constant- α (Q38-39, 41-42)
$\text{Speedup} = E[\text{accepted}]/(k \times c_{\text{draft}} + 1)$	Real speedup — separate step, requires a stated cost assumption (Q39, 41-42)
$\text{Latency}_{\text{budget}} \approx \text{Queue} + \text{Prefill} + \text{Decode} + \text{Network}$	Approximate budgeting tool, NOT exact p99 (Q49-51)

Formula	Meaning
$\text{Cost}_{\text{request}} = \text{GPU_time} \times \text{GPU_cost_rate}$	GPU-time-based production cost model (Q52)

Top Follow-up Q&As

1. Q: Does a faster/bigger GPU always help?

- **A:** Only if the workload is genuinely compute-bound at that hardware's ridge point (Q2) — a memory-bandwidth-bound workload sees no benefit from more compute alone.

2. Q: What's the single biggest real "sounds like an improvement but wasn't" finding across this topic's own notebooks?

- **A:** Quantization's real memory savings producing a net latency *loss* (Notebook 04) — genuine, measured, and honestly reported as-is.

3. Q: How do GQA/MQA, quantization, and batching all relate to the same underlying bottleneck?

- **A:** All three are real, distinct levers on the same roofline-model memory-bandwidth-bound bottleneck (Q2, Q4) — reducing KV-head count, reducing bytes-per-value, and amortizing weight reads across more concurrent work, respectively.

4. Q: Why does this topic repeatedly separate "acceptance/waste percentage" from "real speedup/latency"?

- **A:** Because a real, favorable-looking intermediate metric ($\alpha=0.4$, genuine memory savings) doesn't automatically imply a real, favorable end-to-end outcome — Notebook 04 and Notebook 06 both demonstrated this split directly and honestly.

5. Q: What's the real, honest limitation shared by every simulation in this topic (Q18, Q47)?

- **A:** Real-data-grounded inputs make a simulation more credible than a purely synthetic example, but it remains a simulation of the real mechanism, not an actual implementation or A/B benchmark — a distinction worth preserving explicitly.

6. Q: If you could only track one of the eight observability metrics (Q53), which would you pick and why?

- **A:** None is a safe substitute for the others by design — the question itself is a trap; real production monitoring needs the full set specifically because each catches a distinct real failure mode the others would miss.

7. Q: How does Module 01's roofline framing connect to every later module in this topic?

- **A:** Every later technique targets one side of it directly — FlashAttention/quantization reduce real memory traffic (helping memory-bound ops); batching/speculative decoding increase real

effective arithmetic intensity (amortizing fixed costs across more useful work).

8. Q: What real, general discipline do this topic's own notebooks demonstrate about honest experimentation?

- **A:** Report the actual real result — even a slowdown, an unavailable planned tool, or an implementation bug — rather than adjusting the finding or silently substituting a workaround to match expectations.

9. Q: Why does GQA/MQA appear in both the KV-cache-memory (Q8-9) and quantization (Q25-27) discussions?

- **A:** They're genuinely separate, composable real levers on KV-cache memory — GQA/MQA reduces the *count* of cached vectors; quantization reduces the *bytes per* cached value — and production systems often apply both together.

10. Q: What's the real relationship between PagedAttention (Q13-18) and continuous batching (Q31-36)?

- **A:** Genuinely complementary, not competing — continuous batching's constant slot turnover specifically demands the cheap, frequent (re)allocation PagedAttention's block-based design provides (Q35).